**Politecnico di Milano**
School of Industrial and Information Engineering

**Jan Wangrat**

# Intelligent Job Management

**Final Report**
**COMPUTER SCIENCE AND ENGINEERING**

Supervisor:
**Prof. Danilo Ardagna**
Politecnico di Milano
with Federica Filippini, Ph.D. Candidate

Milano, July 1, 2026

## Abstract

Intelligent Job Management (IJM) is a scheduler for shared GPU clusters that combines profile-aware placement, deadline-driven preemption, and cross-version checkpoint resume. It is an independent reimplementation of the cost-aware, profiling-driven approach of the ANDREAS project (Politecnico di Milano), built for a *heterogeneous* cluster: one FastAPI service drives per-node workers over SSH-tunnelled HTTP, an external GPUspb optimiser handles standard-run placement, and — unlike ANDREAS's uniform Tesla V100 deployment — training jobs migrate between different GPU generations (A40, legacy QuadroP600) across torch versions without losing progress. The report describes the deployment topology, the four-phase scheduling pipeline, the configuration surface, and two end-to-end scenarios on a two-node Polimi cluster that validate the profile sweep, deadline-driven preemption, single- and dual-GPU placement, and cross-node checkpoint migration — all driven coherently by the optimiser's decisions.

## Keywords

GPU scheduling, deep learning, job management, profile-aware optimisation, distributed systems, FastAPI, PostgreSQL, Docker

# Contents

# 1. Introduction

Modern deep-learning research depends on shared GPU clusters in which training jobs compete for a small number of expensive accelerators. Operators are expected to place each job on the hardware that meets its deadline at the lowest cost, despite per-GPU prices that vary by more than an order of magnitude between generations. Manual scheduling quickly becomes the bottleneck, and general-purpose cluster managers such as Kubernetes and SLURM optimise for utilisation and fairness rather than cost: out of the box they model neither the differing energy cost of each GPU generation nor a per-job deadline, and their preemption is priority-based, not deadline-driven — so choosing the cheapest hardware that still meets a job's deadline is left to the operator.

The ANDREAS project [AF+21] at Politecnico di Milano demonstrated that a cost-aware optimiser, fed with profile measurements of each job type on each hardware bundle, can plan placements that beat the typical operator's intuition. ANDREAS is itself designed for heterogeneous hardware — its node configuration and energy-cost model are keyed by GPU type, and it can migrate a job across GPUs — but its ARMIDA testbed was a uniform Tesla V100 cluster dispatched through SLURM, so every node ran the same CUDA and torch stack and a migration never crossed a checkpoint-serialization boundary. The two-node Polimi cluster targeted here is heterogeneous in that second dimension too: an A40 node alongside a legacy QuadroP600 node on different CUDA and torch generations. A job preempted on one node must therefore resume *across torch versions* when it migrates to the other — a case a uniform cluster never exercises — and the scheduler must consume a standalone optimiser (GPUspb [FLC+23, FAAG24]) as an HTTP service rather than embed one.

This report describes Intelligent Job Management (IJM), an independent reimplementation of the ANDREAS approach as a distributed control plane. The control plane is a single FastAPI service that hosts the profiling scheduler, the optimiser client, and the per-node slot semaphores. Each GPU node runs a lightweight worker that owns the local Docker daemon. The two are glued together by plain HTTP over SSH tunnels, so the same code runs on a laptop talking to a remote cluster and on a single-host development machine. IJM integrates the GPUspb optimiser as the external decision maker for standard-run placement. Before any instance is placed as a standard run, the scheduler first executes a small number of profile runs for it on previously-unmeasured (node, GPU-count) cells, so the optimiser is never asked to score an unmeasured bundle. Migration between the A40 and legacy P600 nodes mid-training is what makes such placement worthwhile, and it is not free: a PyTorch checkpoint is normally locked to the version that wrote it, and the two nodes can only exchange one through a shared filesystem. IJM closes both gaps — a deliberately version-tolerant checkpoint format and an rclone-mounted shared store — and asks of each training job only that it honour a small *contract*; the solution and its reasoning are detailed in §2.7 and the contract in Chapter 3.

We validate IJM with two end-to-end scenarios on the two-node cluster, each exercising the full job lifecycle and checking that the system faithfully realises the optimiser's decisions. Scenario 1 (under-subscribed) submits five instances of `cnn_big`, a compute-heavy synthetic CNN: four loose-deadline "patient" jobs and one tight-deadline "urgent" job. It drives the profile sweep, single- and dual-GPU placement, and an urgent job that migrates across nodes — off the legacy P600 onto the A40 — resuming its checkpoint across torch versions. Scenario 2 fills the cluster and mixes types (`cnn_big` pins on A40, slack `lstm-small` patients — a small LSTM — on P600), adding deadline-driven preemption of the cheapest slack work and a

second cross-node migration. Together they confirm that profiling, placement, preemption, and cross-version resume behave correctly and stay consistent with the optimiser's plan. (As a secondary observation, the runs also expose a *horizon-myopia* in the GPUspb `method=RG` proxy — an urgent job parked on slow-but-free hardware — which Appendix A characterises but which does not affect IJM's own correctness.)

Chapter 2 describes the deployment topology, the scheduling pipeline, the state machine, the persistent state, and the invariants that protect the control plane from concurrency hazards. Chapter 3 documents the cluster and job configuration files, the submission API and the trainer contract, and the runtime knobs. Chapter 4 covers testing and evaluation: unit and invariant tests that exercise the control plane's concurrency guarantees under injected faults, and the two end-to-end scenarios on the cluster. Chapter 5 summarises the contributions, the limitations, and the follow-up work that has concrete entry points in the codebase. Appendix A documents the GPUspb proxy's objective and cost model and the horizon-myopia the scenarios surface.

# 2. System Architecture

This section presents IJM's deployment topology, the four-phase scheduling pipeline that drives every dispatch decision, the worker execution lifecycle that runs and stops each container, the job state machine, the persistent state held in PostgreSQL, and the concurrency invariants that keep the control plane consistent under racing API calls and worker notifications. Table 2.1 lists the five processes that make up a deployment, and Table 2.2 the components that run inside the API process.

| Process | Binary / host | Responsibility |
|---|---|---|
| API | `ijm-api` (FastAPI) | HTTP frontend; hosts the scheduling and dispatch loops |
| Worker | `ijm-worker` (FastAPI) | One per GPU node; owns the local Docker daemon, runs the container, emits slot-freed notifications |
| PostgreSQL | `postgres` (a GPU node) | Source of truth for job rows and profile measurements, and the message bus (`LISTEN/NOTIFY`) |
| GPUspb optimiser | external HTTP | Cost-aware standard-run placement (reached via `OPTIMIZER_URL`) |
| SPA | static (Vite) | Operator UI; polls the API every 3–5 s |

**Table 2.1:** IJM's five deployable processes.

| Component | Responsibility |
|---|---|
| `ProfilingScheduler` | Picks the next unmeasured `(node, GPU-count)` cell for profile-owing rows |
| `JobDispatcher` | Acquires the destination node's GPU permit — a per-node semaphore (`NodeSlots`) reconciled with the DB by a drift heartbeat — then POSTs `/jobs/{id}/run` to the target worker |
| Optimizer client | Packages the GPUspb request, parses the returned assignments and preempts |

**Table 2.2:** The components that run inside the API process.

## 2.1 Deployment topology

IJM runs as a small distributed system. The FastAPI *API* runs on a control host — the operator's laptop in our prototype, though nothing stops it from running on a dedicated node or in a cluster — and hosts the `ProfilingScheduler`, the `JobDispatcher`, and the optimiser client, backed by the per-node `NodeSlots` semaphores and the cluster config it loads at startup. A lightweight FastAPI *worker* runs on every GPU node and owns the local Docker daemon. PostgreSQL runs on one of the GPU nodes, deliberately close to the workers (Figure 2.1): the per-epoch status and progress writes from every running container make the worker $\leftrightarrow$ database link the highest-volume traffic in the system, so it should not have to cross the SSH tunnel back to the control host.

Components talk over plain HTTP carried on SSH tunnels. The scheduler addresses each worker by a stable per-node alias rather than by its physical address, and the tunnel maps that alias to the right machine; the same scheduler code therefore runs unchanged whether the control host is a laptop tunnelling into a remote cluster or a node co-located with the workers. The GPUspb optimiser is reached the same way.

Which transport carries what is a deliberate split rather than a matter of convenience. *Commands* — start a container, stop a container — travel API → worker as HTTP requests: each is addressed to one named node and waits on a synchronous reply, so request/response fits, and the `JobDispatcher` is their sole origin. *State changes, and the scheduling wake-ups they imply* — a run reaching a terminal state, a GPU slot coming free, a profile finishing — travel the opposite way, worker → API, as a committed row write paired with a `NOTIFY`; a worker never issues an HTTP call back to the control plane.

PostgreSQL doubles as the message bus, so IJM needs no separate broker. Components coordinate by writing a row and issuing `NOTIFY` on one of two channels in the same transaction — `ijm_schedule` to wake the scheduler, `ijm_slot_freed` when a GPU slot is released — while their peers `LISTEN`. The API wakes its own in-process scheduler the same way: a submission commits its row and `NOTIFY`s rather than calling the scheduler directly, so local and worker-originated wakes share one path. Because each notification commits in the same transaction as the state change it announces, a committed change is never left unsignalled; the watchdogs of §2.2 recover only the rare notification dropped by a connection reset.

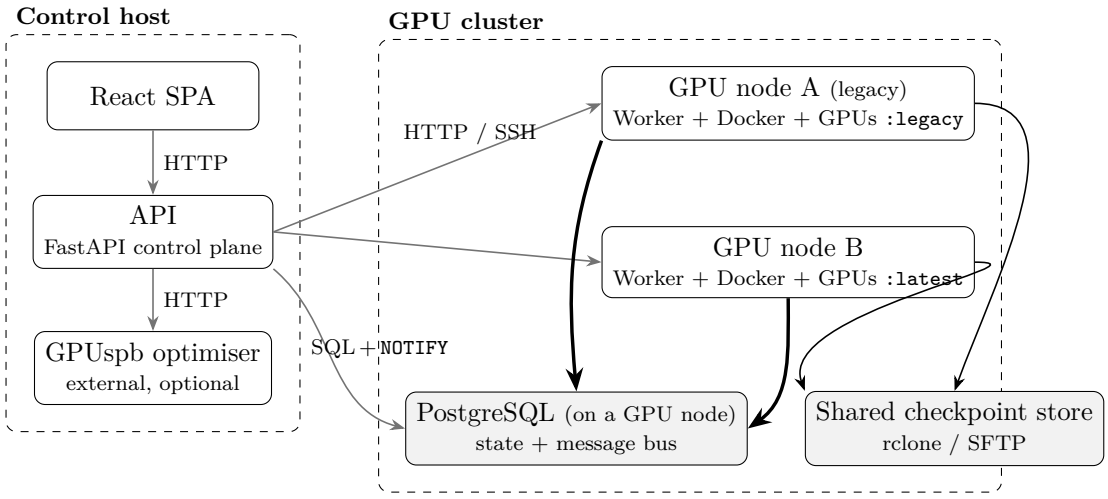


**Figure 2.1:** Distributed prototype, at a glance. Each arrow points from the *initiator* of a connection to the service it calls; a single head is request/response, *not* a two-way channel, and line thickness shows relative volume, not direction. The API calls the workers (HTTP `/run`, `/stop`), the optimiser (one POST per pass), and PostgreSQL (SQL; it also holds the `LISTEN`) — all over SSH tunnels. Neither the optimiser nor PostgreSQL ever calls back: a worker reports a finished run or a freed slot by committing a row and `NOTIFY`ing, which reaches the API through the database, never by an HTTP call to the control plane. The thick worker → PostgreSQL arrows are the *heavy* per-epoch status and progress writes — the reason PostgreSQL is co-located with the workers on a GPU node rather than placed by the control host. The training containers read and write the shared checkpoint store, so a checkpoint written on one node is visible from the other.

## 2.2 Scheduling pipeline

The pipeline is shaped by one governing assumption: *profiling is always worth its cost.* A deep-learning job runs for hours; profiling one GPU configuration takes minutes, so leaving a configuration unmeasured risks missing the placement that would have saved hours — or a hard deadline. The scheduler therefore treats profiling as privileged work that may preempt running jobs, not best-effort measurement deferred until the cluster falls idle. `PROFILING_CONFIGS_PER_JOB` (default 2) sets how many configurations an instance must measure before it is handed to the cost-aware optimiser for standard placement.

A single pass — `_schedule_waiting_jobs` in `app.py` — handles every dispatch decision, invoked whenever a job is submitted, a profiling run finishes, or a slot is freed; a 60-minute watchdog (`_queue_watcher`) is only a safety net for missed notifications. Wake-ups are coalesced through a single `asyncio.Event` and rate-limited to one pass every five seconds, so the cluster can apply a plan before the next is computed. The limit defers, never drops: the event latches and every pass reads committed rows, so a wake-up arriving mid-interval — a job finishing, say — is serviced by the next pass, at worst one interval later.

### 2.2.1 The four phases of a scheduling pass

The pass runs four phases, all guarded by a single `schedule_lock` except the optimiser's HTTP call (phase 2), which is made outside the lock so its timeout cannot stall concurrent scheduler work. The optimiser runs *before* the profiling phases by design: it is a slow external solver, so it reasons over one snapshot taken at the start of the pass, and the cheap profiling phases run afterwards, reading its assignments so they preempt *around* that plan. This does not subordinate profiling — a profile-owing row is invisible to the optimiser, and when no slot is free phase 4 evicts a standard job and reserves the freed slot — so profiling keeps precedence regardless of order. The four phases are:

1. **Reset stuck assignments.** A row can be left `QUEUED` with `assigned_node` set but no container behind it — a dispatch the optimiser (or a pending migrate) planned that never landed: the image pull failed, the worker was unreachable, or the API restarted between writing the assignment and posting `/run`. Left alone it pins the row to a node forever, holding a slot the optimiser believes is occupied. This phase clears any `QUEUED` row stale for $\geq 30$ s with no live dispatch task — well above a healthy `/run`'s sub-second latency — and reclaims the leaked slot for a later phase to re-place. In steady state it resets nothing.

2. **Optimiser pass.** Bounded only by a 30 s timeout, the optimiser call runs outside `schedule_lock` (as above); the lock is re-acquired only to apply the returned assignments, through a conditional `UPDATE ...WHERE assigned_node IS NULL` — a suggestion for a row claimed since the call simply does not match — while collecting the optimiser's preempt list. The optimiser only ever scores rows with `is_profiling_run = FALSE`; rows still owing a profile are invisible to it.

3. **Profile placement.** For `QUEUED` rows still owing a profile, the `ProfilingScheduler` atomically reserves the instance's outstanding unmeasured `(node, GPU-count)` cells — up to its `PROFILING_CONFIGS_PER_JOB` budget — and dispatches one this pass by best fit on remaining capacity; the rest follow as slots free. This enforces *profile-always*: an instance must finish its profile runs before it is a candidate for standard placement.

4. **Profile-preempt.** If placement leaves a profile-owing row without a slot, the scheduler evicts the cheapest *minimal* set of `RUNNING` jobs that frees enough GPUs for the next unmeasured configuration. On each profiling-capable node whose *total* capacity could host it, it ranks the `RUNNING` jobs cheapest-first (priority ascending, then progress ascending) and picks the smallest victim subset covering the shortfall — a higher-ranked victim only when no cheaper set covers, and never an over-large one (it will not evict a small job and then a larger one that alone would have sufficed). A `PROFILING` row is never a victim. The first node with such a set wins; its victims are evicted and the freed slots claimed next pass.

### 2.2.2 Selecting which configurations to profile

Which configurations an instance profiles is deterministic, not opportunistic. The valid configurations for a job type are the GPU bundles common to the profiling and production nodes, sorted by total GPU count and then by GPU-type name; an instance profiles the first `PROFILING_CONFIGS_PER_JOB` (default 2) of them that are still unmeasured for its type. Reservations are atomic and shared: each `(job_id, gpu_config)` cell is claimed under the unique index of §2.5, so two instances of the same type never measure the same cell and one instance's measurement is reused by all the others. No separate "profiling finished" flag is kept in sync: an instance is in profiling mode *iff* it still owns an unmeasured claim, so once its budget is spent — or no unmeasured configuration remains — `is_profiling_run` derives as `FALSE` and the row falls through to the optimiser.

### 2.2.3 Dispatching the plan and releasing slots

Once the lock is released, the plan is executed asynchronously: each preempt and each dispatch runs as an independent task, so a slow stop on one node never delays placements on another. Before launching a job, the dispatcher acquires a permit from the destination node's `NodeSlots` semaphore, whose count is that node's GPU capacity; the permit bounds concurrent jobs per node and arbitrates the last free slot.

Permits are released asynchronously, because a job's GPUs are not free the instant the API decides to stop it — only once the worker has killed the container, drained its logs, and written a checkpoint. The worker therefore signals each release, and the API returns the permit at that moment, keeping its in-memory count in step with actual occupancy. Each release also records *why* it happened, because the cause determines whether the scheduler should re-plan. A release triggered by an event outside the current plan — a job finishing on its own, an operator stopping a job, or the cleanup of an orphaned container — frees capacity the optimiser did not foresee, and so wakes a fresh scheduling pass. A release caused by the scheduler's *own* in-flight preempt is the one exception: the plan that issued the preempt already contains the dispatch that will reclaim the freed slot, so re-running the optimiser at that point would let it rewrite a plan it is still executing. Only that case suppresses the re-plan, and doing so is safe because independent watchdogs — a drift heartbeat, the stuck-dispatch reaper, and a periodic queue watcher — re-wake the scheduler if a planned dispatch ever fails to land.

### 2.2.4 Walking a job through the pipeline

The figures follow a job through that lifecycle in order: Figure 2.2 submission and the first (profiling) pass, Figure 2.3 the cross-instance profile cycle, Figure 2.4 the optimiser-driven scheduling round that preempts and migrates, and Figure 2.5 a standard run.

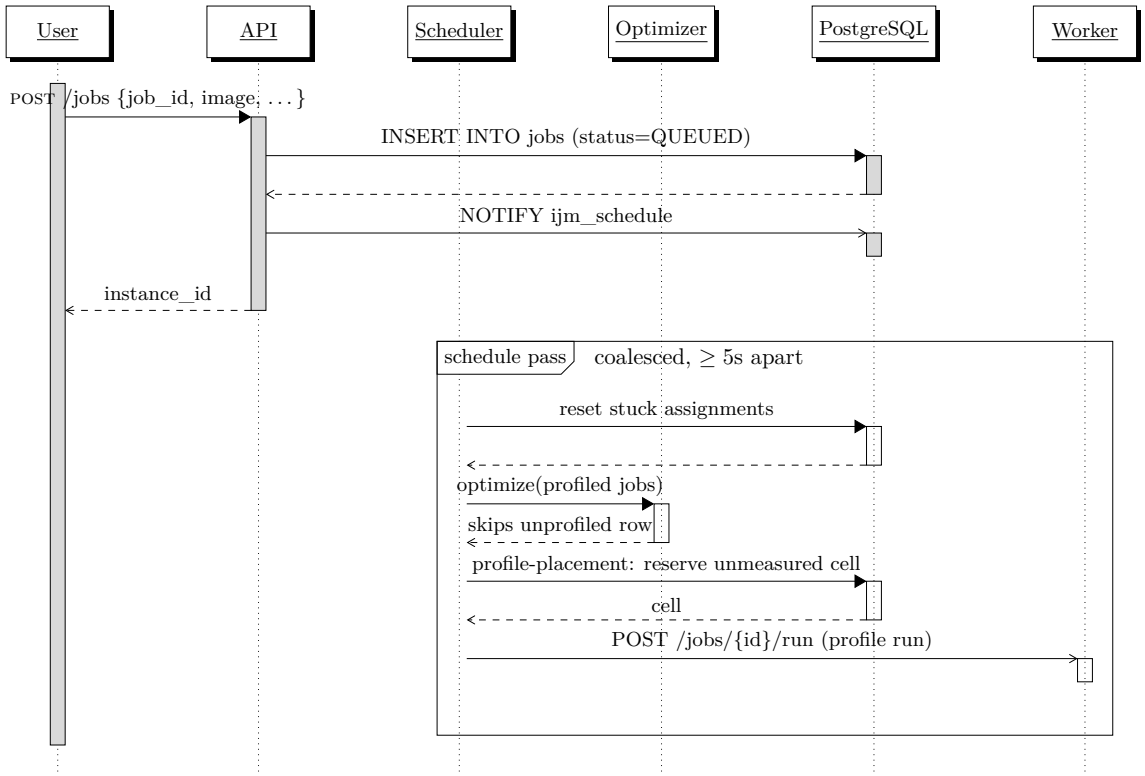


**Figure 2.2:** Submission and profiling. The API writes the QUEUED row and notifies the scheduler. A freshly submitted job still owes a profile run, so the optimiser pass skips it (it scores only profiled rows); phase 3 reserves an unmeasured `(node, GPU-count)` cell and dispatches the profile run, the dispatcher acquiring the node's `NodeSlots` permit before posting `/run`. A job submitted with no configuration left to profile skips straight to the optimiser, which places it as a standard run (Figure 2.4). The optimiser call sits outside the schedule lock so its 30 s timeout cannot block other passes.

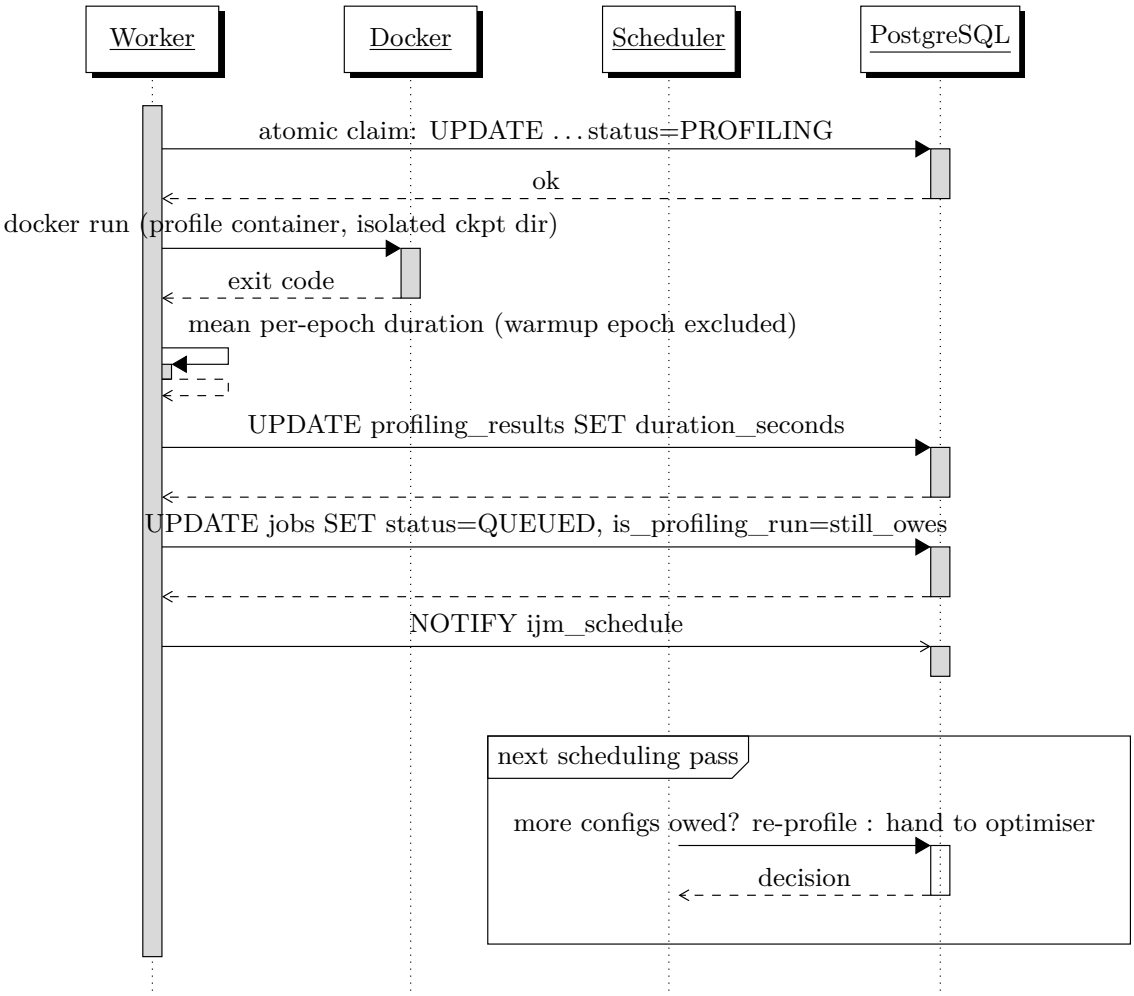


**Figure 2.3:** Profiling cycle. The `NOTIFY` re-queues the row, and the next pass reads the `is_profiling_run` flag: while the instance still owes a configuration it is profiled again, and once all its `PROFILING_CONFIGS_PER_JOB` (default 2) configurations are measured it becomes a standard-run candidate, handed to the optimiser (Figure 2.4). Measurements are keyed by the (job-type, GPU-config) pair and shared across submissions of the same type; the per-instance claim row guards against duplicate measurements. The worker records the result and re-queues the row in a single transaction (`worker/profiling.py:handle_complete`).

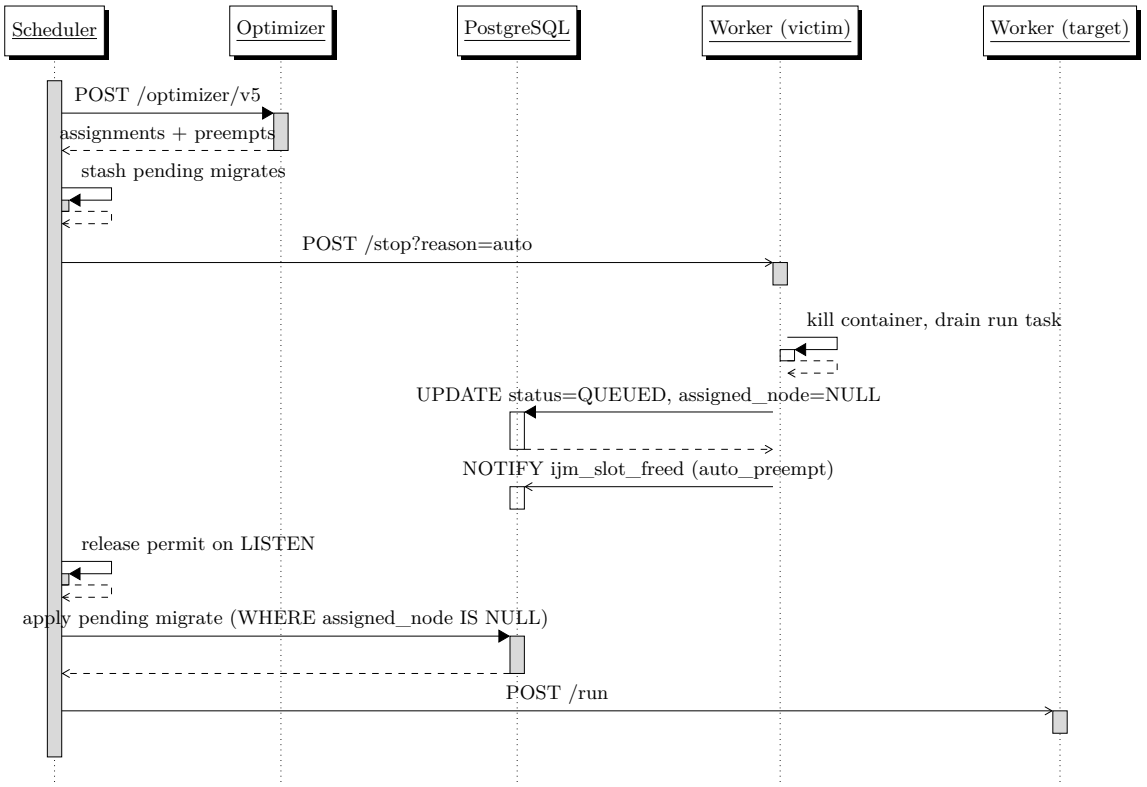


**Figure 2.4:** Optimiser-driven preempt and migrate. The API trusts the GPUspb plan: any job in `currentScheduling` that the optimiser drops or moves is preempted with `reason=auto`, clearing its assignment. Because an `auto_preempt` release deliberately does *not* wake a fresh optimiser pass, a moved job's new placement is stashed in `state.pending_migrates` and re-applied by the slot listener (conditional `UPDATE ...WHERE assigned_node IS NULL`) once the source `/stop` has drained the row to QUEUED.

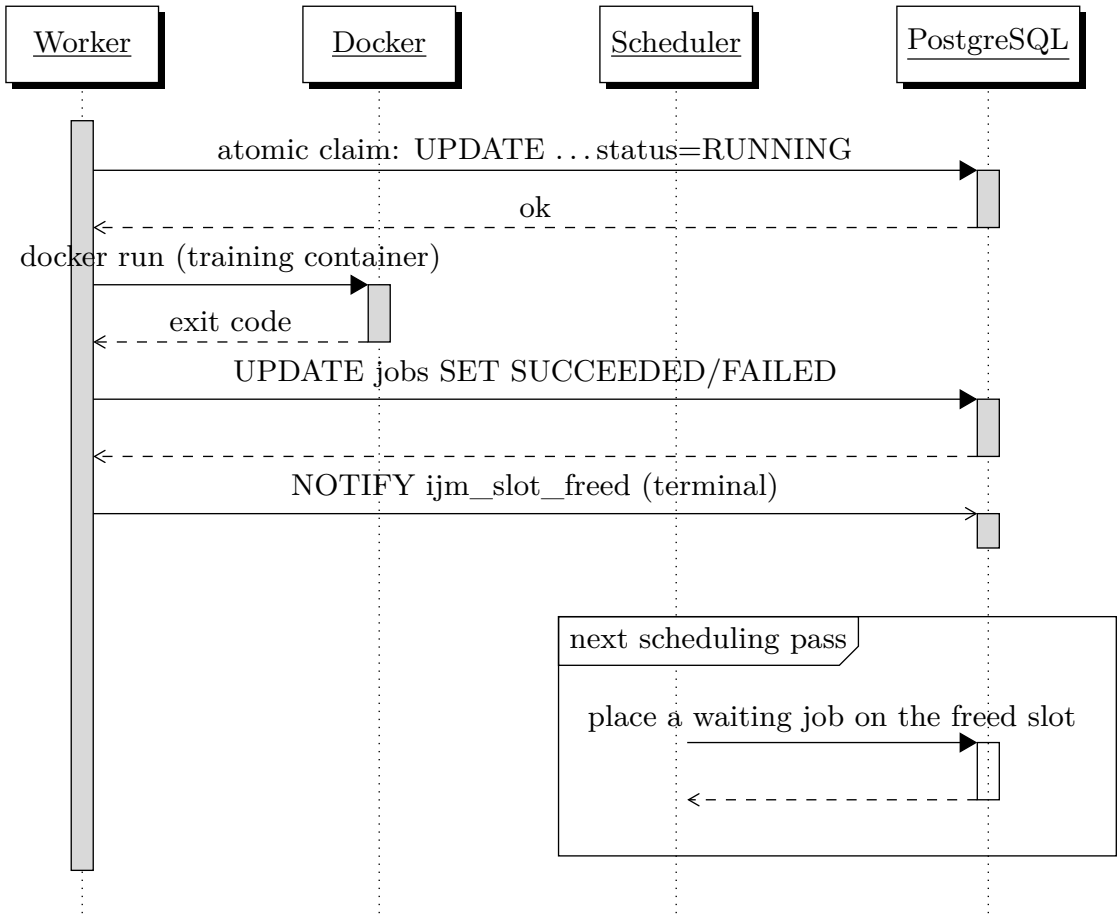


**Figure 2.5:** Standard run. The worker performs an atomic conditional `UPDATE ...WHERE status = ANY(RUNNABLE_STATUSES)` so concurrent stops or duplicate dispatches cannot race a row into `RUNNING`. On exit the worker fires a single `ijm_slot_freed` (reason `terminal`) in the same transaction as the status flip; the slot listener releases the permit and wakes the scheduler, so the terminal path needs no separate `ijm_schedule`.

## 2.3 Worker execution lifecycle

The scheduling pass decides *where* a job runs; the worker's run task (`_run_job`) owns *how* it runs and how it stops. The task moves through four phases (Table 2.3), and every database transition is a conditional `UPDATE`, so a racing `/stop`, the stuck-dispatch reaper, or a duplicate dispatch cannot corrupt the row.

| Phase | Action | Invariant it enforces |
|---|---|---|
| 1. Claim | Atomic `UPDATE ...status=RUNNING/PROFILING WHERE status` ∈ RUNNABLE `AND assigned_node=self` | A `/stop` or reaper that already moved the row makes the claim miss, so no container starts for a re-placed row. |
| 2. Launch | Pick distinct physical GPU indices, prepare the per-job checkpoint/runs directories, `docker run` | Distinct indices stop two 1-GPU jobs both landing on GPU 0; an in-process placeholder is set *before* any `await`, so a `/stop` cannot slip into the window before the container exists. |
| 3. Stream | Parse `Epoch x/y` from container stdout; flush the latest value through a once-per-second background writer | The progress `UPDATE` is conditional on a matching `status` and `container_name`, so a reassigned row no-ops it and stops the stream. Coalescing replaces a per-epoch round-trip that fell behind over the SSH-tunnelled connection. |
| 4. Complete | Write the terminal status and fire one `ijm_slot_freed`; or, if the row was migrated or already stopped, record only the exit code | A kill-induced exit 137 never clobbers the new owner's RUNNING; a failed run also drops its outstanding profile claims so their cells are not left reserved. |

**Table 2.3:** The worker run task's four phases — distinct from the scheduling pass's four phases above: that pass plans, this task executes.

Within the one worker process on the node, a `/stop` and its target run task are separate coroutines that must not race. The handler tags the row in an in-process `stopping` map, waits for the run task to publish its container (`dispatch_ready`), kills it, and only *after* awaiting the run task's full drain writes the final status and frees the slot (Figure 2.6). The run task's own completion phase, seeing the `stopping` tag, records just the exit code, so the slot is freed exactly once. The same hazard across nodes is closed on the API side: `dispatch_with_slot` awaits any in-flight `/stop` on the target node before posting `/run`, so a migration's new container never starts before the evicted one's CUDA context has released its GPU.

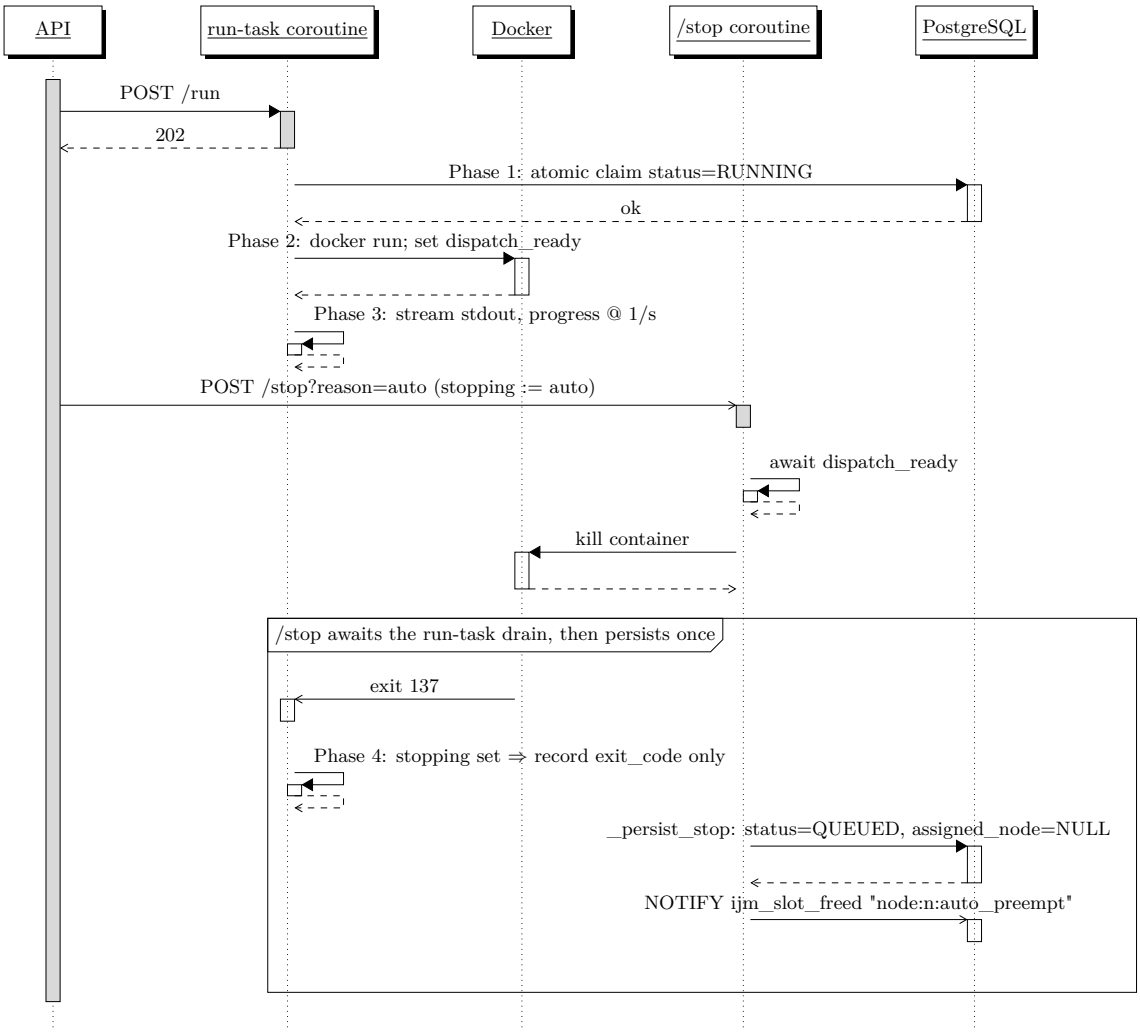


**Figure 2.6:** Worker execution and a scheduler stop, all *inside one worker process*. The two middle lifelines are not two workers: they are the node's single worker acting as two concurrent coroutines — the run task spawned by `POST /run` and the handler spawned by `POST /stop` — drawn apart only because they run at the same time. The run task owns the `docker run` subprocess, so it (not `/stop`) is the coroutine that observes the container's `exit 137`; `/stop` merely kills the container, then awaits the run task's drain before persisting. The two coroutines synchronise through the in-process `dispatch_ready` and `stopping` maps plus that drain await, so the slot is freed exactly once and by exactly one path. A `reason=auto` stop returns the row to QUEUED with its assignment cleared (a `reason=user` stop lands on PREEMPTED instead); the resulting `auto_preempt` notify is the one the slot listener swallows (§2.2).

## 2.4 State machine

Table 2.4 lists every transition a job row can take. All transitions are driven by atomic conditional `UPDATE`s, so concurrent stop, dispatch, and completion paths cannot clobber each other's work.

| From | To | Trigger |
|---|---|---|
| QUEUED | PROFILING / RUNNING | Worker Phase 1 atomic claim |
| PROFILING | QUEUED | Profile run finished; `is_profiling_run` flips to `false` |
| PROFILING | FAILED | Profile container exited non-zero |
| RUNNING | SUCCEEDED / FAILED | Container exit |
| RUNNING | PREEMPTED | User `POST /jobs/{id}/stop?reason=user` |
| RUNNING | QUEUED | Scheduler preempt (`reason=auto`): clears `assigned_node` for the next pass |
| PREEMPTED / FAILED | QUEUED | `POST /jobs/{id}/resume` |

**Table 2.4:** Job state transitions.

Two distinct stop reasons separate *user-initiated* preemption from *scheduler-initiated* preemption. A user stop is sticky: PREEMPTED stays put until the user explicitly resumes. A scheduler stop returns the row to QUEUED with a cleared assignment, so the next pass can place it elsewhere automatically. The worker enforces this distinction and ignores stale `/stop` calls aimed at a row that has since been re-assigned to another node.

## 2.5 Persistent state

Two PostgreSQL tables hold every fact the scheduler reads or writes; both are created idempotently on startup from `backend/schema.sql`. Each row of `jobs` is one submitted job *instance* (the term "row" below always means one such instance), while `job_id` names the job *type* that many instances can share.

```
CREATE TABLE jobs (
    id                  TEXT PRIMARY KEY,      -- instance id (one submission)
    job_id              TEXT NOT NULL,         -- job *type*; keys profiling
    image               TEXT NOT NULL,         -- container image
    command             JSONB NOT NULL,        -- container argv
    script_path         TEXT,                  -- optional training script
    directory_to_mount  TEXT,                  -- optional host dir mounted in
    status              TEXT NOT NULL,         -- lifecycle state (see 2.4)
    created_at          TIMESTAMPTZ NOT NULL,  -- submission time
    updated_at          TIMESTAMPTZ NOT NULL,  -- last change (reaper input)
    container_name      TEXT,                  -- docker name while running
    exit_code           INT,                   -- container exit code
    progress            TEXT,                  -- latest "Epoch x/y"
    priority            INT DEFAULT 3,         -- lower value preempted first
    deadline            TIMESTAMPTZ,           -- drives optimiser urgency
    batch_size          INT,                   -- training batch size
    epochs_total        INT,                   -- epochs in a full run
    profiling_epochs_no INT,                   -- epochs per profile
    assigned_node       TEXT,                  -- placed node; NULL=unplaced
    assigned_gpu_config JSONB,                 -- chosen bundle {A40:2}
    is_profiling_run    BOOLEAN DEFAULT FALSE -- TRUE=profile run (hidden)
);

CREATE TABLE profiling_results (
    id               TEXT PRIMARY KEY,     -- measurement/claim row id
    job_id           TEXT NOT NULL,        -- job *type* measured
    instance_id      TEXT,                 -- instance that claimed it
    gpu_config       JSONB NOT NULL,       -- measured bundle {A40:1}
    node_id          TEXT NOT NULL,        -- node measured on
    duration_seconds FLOAT,                -- mean per-epoch s; NULL=none
    created_at       TIMESTAMPTZ NOT NULL -- claim time
);
```

Two columns of `jobs` are load-bearing for the scheduling pass above: `assigned_node` is the target the optimiser pass writes under `WHERE assigned_node IS NULL`, and `is_profiling_run` is the flag that makes a row invisible to the optimiser (phase 2) and visible to the `ProfilingScheduler` (phases 3–4).

A unique index on `(job_id, gpu_config)` makes the cross-instance profile claim atomic. Profile measurements are keyed by the *job type* (`job_id`) and the GPU configuration, so two concurrent submissions of the same type race to own each unmeasured cell; the loser's INSERT trips the unique constraint and the winner's measurement is shared by every subsequent instance of that type.

## 2.6 Concurrency invariants

The control plane relies on three invariants, each defended in code.

**No GPU oversubscription.** `NodeSlots` is a *counting* semaphore per node: its capacity is the node's total GPU count, and every dispatch calls `acquire(node_id, n)` to reserve `n` permits. It bounds *how many* GPUs are concurrently in use on a node, not *which* ones — the concrete physical GPU indices a container binds to are chosen by the worker at launch (Phase 2, distinct indices), not by `acquire`. Matching the *type* of GPU is handled upstream, at placement: the optimiser and the `ProfilingScheduler` only put a `(node, gpu_config)` where that node has free GPUs of that type, computed per type from the database (`get_node_gpu_usage`). In the supported cluster each node exposes a single GPU type, so the per-node permit count and the per-type free capacity coincide; a mixed-type node would rely on that placement check, not on the counting semaphore, to avoid per-type oversubscription. The semaphore is not a second source of truth — the database is, and the atomic row claim of the next invariant already stops two containers sharing one *job*; it earns its keep because neither of those bounds *per-node* concurrency once the schedule lock is released and dispatch tasks run in parallel. `acquire` caps concurrent dispatches at the node's capacity and, crucially, lets a dispatch *block* until a slot frees — a migrate waiting for an in-flight preempt to vacate its GPU — which a stateless database check cannot do without polling. On startup the semaphores are reconciled from the database: only rows in RUNNING or PROFILING pre-acquire their permits, since a QUEUED row with an assigned node has no live container yet (its dispatch task acquires the permit just before posting `/run`). A periodic drift watcher recomputes the authoritative count at the `IJM_DRIFT_HEARTBEAT_S` cadence (default 15 s) and corrects any divergence.

**No duplicate containers per job instance.** Both the worker (Phase 1 claim, `running_jobs[job_id]` check) and the API (`dispatch_tasks[instance_id]` deduplication) refuse a second concurrent dispatch for the same instance. The reaper that resets stuck QUEUED rows cancels any live dispatch task instead of double-releasing its permit.

**Profiling is sacred.** A profiling run is never *offered* for preemption in the first place: the optimiser request and its `currentScheduling` are built only from rows with `is_profiling_run = FALSE` (§2.2), and a fully profiled `RUNNING` job never reverts to a profiling run, so the preempt list the optimiser returns can never name a measurement in flight. The auto-preempt path (`_preempt_and_release`) re-reads the flag anyway and, if it ever sees `is_profiling_run = TRUE`, refuses and logs a caller bug — a fail-loud assertion guarding the invariant, not the primary mechanism. Profile-preempt (phase 4) likewise considers only `status='RUNNING'` victims, never `PROFILING` ones. Only an explicit user stop can cancel an in-flight measurement.

These three invariants are not only defended in code but verified empirically under induced faults: Chapter 4 (§4.3.2) races the placement and Phase-1 claims, collides the reaper

with an in-flight claim, and forces the in-memory slot count to diverge from the database in both directions, then shows the drift heartbeat reconciling in real time with no oversubscription.

## 2.7 Stoppable, resumable, cross-node training

A job is *stoppable* because the trainer persists its state after every epoch, and *resumable* because every (re)start loads that state. `BaseTrainer.train` writes `latest.pt` at the end of each epoch through a temp-file-and-rename, so the on-disk checkpoint is always a whole epoch, never a half-written file. Stopping a job is therefore just killing its container (§2.3): the in-progress epoch is discarded, every completed epoch is durable. The next dispatch — on the same node or another — reconstructs the trainer, which loads `latest.pt`, restores the model, optimiser, and `current_epoch`, and continues the loop (Figure 2.7). The scheduler does keep a `progress` string per row, but only as a display-and-heuristic cache (the latest `Epoch x/y`); it is allowed to lag and is never read to decide resumption. Where a (re)started job continues from is determined solely by `latest.pt` — the checkpoint file is the single source of truth for how far a job has actually run.

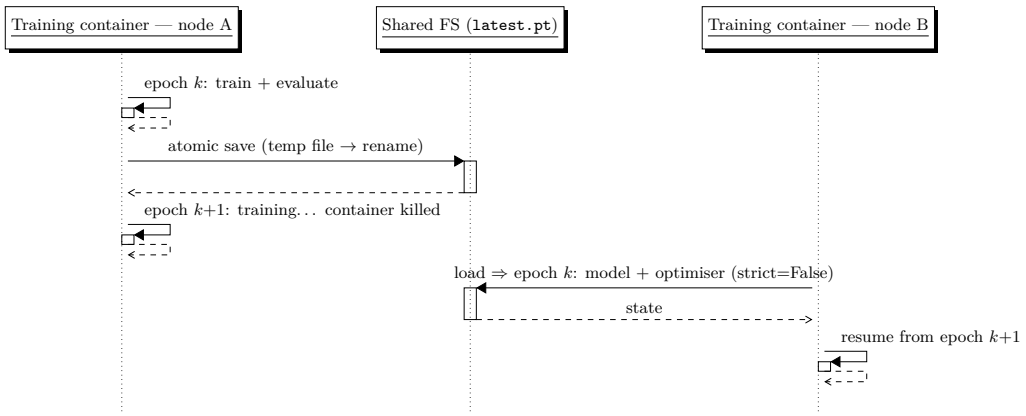


**Figure 2.7:** Stop and resume across nodes. Each lifeline is the training *container* (the trainer process the worker launched via `docker run`) on its node. The container checkpoints after each epoch with a temp-file-and-rename, so `latest.pt` is never half-written; a stop kills it mid-epoch, and the next run — here on a different node, possibly a different torch version — loads `latest.pt` and continues, so only the interrupted epoch $k{+}1$ is recomputed.

### 2.7.1 The shared checkpoint filesystem

Cross-node resume has a hard prerequisite: a checkpoint written on one node must be byte-for-byte readable on every other node. IJM meets it with a single shared `data/` tree that every node FUSE-mounts via rclone over SSH/SFTP (`rclone mount ... -vfs-cache-mode writes`); the worker bind-mounts the per-job slices of that tree into each container (Table 2.5). Because all nodes see the same files, the `latest.pt` written on the A40 node is exactly the one the P600 node loads when the job migrates. Without one shared namespace the next dispatch would find no checkpoint and silently restart from epoch 0 — so the mount is not an optimisation but the substrate the whole stop/migrate/resume story rests on.

| Shared path | In container | Contents |
|---|---|---|
| `data/checkpoints/{job_id}/` | `/checkpoints` | `latest.pt`; a `.profiling/` subdir isolates profile runs |
| `data/runs/{job_id}/` | `/runs` | captured `output.log` and TensorBoard artifacts |
| `data/shared/data/` | `/runs/data` (ro) | dataset cache (MNIST, CIFAR-10), staged once |
| `data/pg/` | — | PostgreSQL volume; node-local, *not* on the rclone mount |

**Table 2.5:** The shared `data/` tree and where each slice mounts.

A networked, multi-writer mount breaks three assumptions that hold on a local disk; each forces a specific measure:

- **Partial reads.** A peer must never see a half-written checkpoint, so the trainer writes a temp file and atomically `rename`s it over `latest.pt` — on the mount the rename publishes the whole file or nothing.
- **`mmap` over FUSE.** `torch.load` memory-maps and seeks the file, which an rclone/SFTP mount rejects with `EPERM`; the loader therefore pre-reads the bytes into memory (`io.BytesIO(path.read_bytes())`) and deserialises from there.
- **Cross-UID ownership.** Rootful Docker on one node and rootless on another map container-root to *different* host UIDs, so a `0600` file written on one node is unreadable through SFTP on the other. The worker pins `-user <uid>:<gid>` to the host owner of `data/`, so every node writes checkpoints as the same user.

### 2.7.2 Surviving a torch-version change

A migration can move a job between nodes on *different* PyTorch builds — the A40 node on PyTorch 2.6 (CUDA 12.4) and the legacy `matemagician` node, pinned to driver 418.x and therefore to PyTorch 1.5.1 on Python 3.7. A checkpoint is normally version-locked, so resume across that gap is not free; four incompatibilities each have to be neutralised, for a concrete reason:

- **Serialization format.** PyTorch switched `torch.save` to a zip container at 1.6, which 1.5.1 cannot read. The trainer always saves in the old tar+pickle format (`_use_new_zipfile_serialization=False`, with a fallback for torch<1.6 where the kwarg does not exist), so either build loads the file.
- **Load-API drift.** The loader tries `weights_only=True` (added in torch 1.13) and falls back when absent, and always passes `map_location=device` so tensors land on the target GPU regardless of where they were saved.
- **State-dict keys.** Parameter naming and `DataParallel` `module.` prefixes drift between versions, so the model loads with `strict=False` — tolerating key differences while still re-raising on a real tensor-shape mismatch, which would mean the weights are genuinely incompatible.
- **Optimiser state.** Adam's step counters and momentum buffers are the most version-fragile part, so their load is wrapped in `try/except`: on failure Adam restarts

from scratch, losing only a few epochs of learning-rate warmup — invisible at scenario timescales — while the model weights, which actually matter, are kept.

One precondition makes all of this reachable: the trainer module must *import* under Python 3.7 in the legacy image, so `runtime/base.py` uses `from __future__ import annotations` to keep modern type-hint syntax from being evaluated at import time. Together these let `cnn_big` migrate between a modern A40 and a legacy P600 mid-training without losing progress.

Two worker-side environment variables let the same `nodes_config.json` drive heterogeneous nodes. `WORKER_GPU_MODE` selects how Docker exposes GPUs to the training container: `runtime` for the standard NVIDIA runtime, `cdi` for the rootless daemon on `polimi-gpu`, and `none` for a CPU-only sandbox. `IMAGE_TAG_OVERRIDE` rewrites the image tag, which is how `matemagician` swaps `:latest` for `:legacy`.

# 3. Configuration and Job Submission

IJM is configured at three layers: a pair of JSON files that describe the cluster, an HTTP submission API for jobs, and a handful of runtime environment variables. This section documents each in turn.

## 3.1 Cluster configuration

The cluster topology is loaded at startup from two JSON files, whose paths are configurable via environment variables.

### 3.1.1 Nodes (`nodes_config.json`)

Each entry describes a compute node and its GPU resources. The example below is the deployment used throughout Chapter 4 (`config/nodes_config.tunnel.json`); the file checked in at `config/nodes_config.json` is a mock cluster used by the unit tests.

```
[
  {
    "id": "matemagician",
    "isForProfiling": true,
    "workerUrl": "http://localhost:8001",
    "cost": 0.10,
    "resources": [
      { "gpu_type": "QuadroP600", "gpu_count": 2 }
    ]
  },
  {
    "id": "polimi-gpu",
    "isForProfiling": true,
    "workerUrl": "http://localhost:8002",
    "cost": 0.30,
    "resources": [
      { "gpu_type": "A40", "gpu_count": 2 }
    ]
  }
]
```

| Field | Type | Description |
|---|---|---|
| `id` | string | Unique node identifier |
| `isForProfiling` | bool | If `true`, the node can host profiling runs. Production jobs may still land on it; profiling jobs may only land on it |
| `workerUrl` | string | HTTP base URL of the node's worker, reached through an SSH tunnel. The API dispatches by posting `/jobs/{id}/run` and `/jobs/{id}/stop` here; every node must be backed by a worker |
| `cost` | float | Idle cost of the node ($/h), used by the optimiser |
| `resources` | array | GPU resources attached to the node |
| `resources[].gpu_type` | string | GPU model name (e.g. `A40`, `L40S`, `QuadroP600`) |
| `resources[].gpu_count` | int | Number of GPUs of this type |

**Table 3.1:** Node configuration fields.

GPU type and count are declared explicitly rather than auto-detected. This admits mixed-GPU nodes that ANDREAS did not support (e.g. a node with both A40 and L40S GPUs), and it lets the operator declare nodes that are not physically attached to the API host — the API only ever talks to workers over HTTP.

### 3.1.2 Energy costs (`gpu_energy_costs.json`)

A lookup table that maps a GPU type and bundle size to an hourly energy cost ($/h). The scheduler consults this table when scoring candidate placements.

```
{
  "A40":         { "1": 0.30, "2": 0.55, "4": 1.05 },
  "L40S":        { "1": 0.40, "2": 0.75, "4": 1.40 },
  "QuadroP600": { "1": 0.03, "2": 0.055 },
  "Blackwell":  { "1": 1.00, "2": 1.85, "4": 3.50 }
}
```

## 3.2 Job submission

**Endpoint:** `POST /jobs`

### 3.2.1 Request body

```
{
  "job_id": "LSTM-small",
  "dockerImage": "ijm-lstm-small:dev",
  "scriptPath": "train.py",
  "directoryToMount": "/data/my-project",
  "Priority": 3,
  "deadline": "2026-03-20T18:00:00",
  "batchSize": 64,
  "profilingEpochsNo": 3,
  "epochsTotal": 20
}
```

| Field | Type | Required | Description |
|---|---|---|---|
| `job_id` | string | yes | Reusable job type identifier; profile measurements are correlated across submissions sharing the same `job_id` |
| `dockerImage` | string | yes | Docker image to execute |
| `scriptPath` | string | no | Path to the training script (overrides Docker CMD with `python3 <scriptPath>`) |
| `directoryToMount` | string | no | Host directory persisted on the job row (ANDREAS-compatibility field); not mounted by the current worker, which mounts only checkpoints, the runs directory, and the shared dataset cache |
| `command` | string[] | no | Command and arguments (defaults to the image entrypoint; overridden by `scriptPath`) |
| `Priority` | int 1–5 | no | Job priority (default 3) |
| `deadline` | datetime | no | Job deadline (ISO 8601) |
| `batchSize` | int | no | Batch size passed as the `BATCH_SIZE` env var |
| `profilingEpochsNo` | int $\geq 1$ | no | Epochs per profiling run (default 3) |
| `epochsTotal` | int $\geq 1$ | no | Total training epochs (default 20) |

**Table 3.2:** Job creation request fields.

Each submission receives a unique UUID instance ID (`id`) for lifecycle tracking, separate from the `job_id` type identifier. When `scriptPath` is provided the container runs `python3 <scriptPath>`; otherwise the image's default CMD is used. Unlike ANDREAS, `Priority` and `deadline` are optional, and an additional `command` field gives flexibility for non-standard workloads.

### 3.2.2 Response (201 Created)

The response carries the full job object, including the scheduler's initial decision: assigned node, GPU configuration, and whether the first run will be a profile or a standard execution.

```
{
  "id": "a1b2c3d4-...",
  "job_id": "LSTM-small",
  "image": "ijm-lstm-small:dev",
  "command": ["python3", "train.py"],
  "status": "QUEUED",
  "is_profiling_run": true,
  "assigned_node": "polimi-gpu",
  "assigned_gpu_config": { "A40": 1 },
  "priority": 3,
  "epochs_total": 20,
  "profiling_epochs_no": 3,
  "created_at": "2026-03-18T10:00:00Z",
  "updated_at": "2026-03-18T10:00:00Z",
  ...
}
```

### 3.2.3 Job image requirements (the trainer contract)

The fields above describe what the API *stores*; this subsection is the *specification* a container author implements so a job is not merely runnable but *stoppable, resumable, and migratable*. A job image is a container whose entrypoint instantiates a subclass of `BaseTrainer` (`runtime/base.py`) and calls `.train()`; subclassing *is* the contract, and in return the stop/resume/migrate machinery of §2.7 comes for free.

**What you implement.** A subclass provides up to three hooks (Table 3.3). For a dataset bundled with the runtime you implement only `_create_model`: subclass one of the ready-made mixins — `MNISTTrainer`, `MNISTSequenceTrainer`, or `CIFAR10Trainer` — which already supply the other two.

| Hook (you override) | Signature and contract |
|---|---|
| `_create_model` | `(self) -> nn.Module`. Return the model. `BaseTrainer` moves it to the device and, on a multi-GPU node, wraps it in `nn.DataParallel` automatically. |
| `_load_datasets` | `(self) -> tuple[Dataset, Dataset]`. Return `(train, test)` *datasets* — not `DataLoaders`; `BaseTrainer` builds the loaders itself (train: `BATCH_SIZE`, shuffled, `drop_last`; test: `min(256, BATCH_SIZE)`). |
| `_preprocess_batch` | `(self, images, labels) -> tuple[Tensor, Tensor]`. Reshape or move a batch before the forward pass (e.g. `images.squeeze(1)` to feed an LSTM); return `(inputs, labels)`. |

**Table 3.3:** The three trainer hooks. With a bundled dataset only the first is required — the mixin supplies the other two.

**What you get for free.** In exchange for honouring the hooks, `BaseTrainer` supplies the behaviours the scheduler relies on (Table 3.4); the mechanism behind each is detailed in §2.7.

| The image must... | How `BaseTrainer` does it | Why IJM needs it |
|---|---|---|
| checkpoint after every epoch | writes `/checkpoints/latest.pt` via a temp-file-and-rename | the checkpoint is the *only* record of progress, so a job can be killed at any instant |
| resume on start | loads `/checkpoints/latest.pt` (model, optimiser, epoch) before training | makes (re)dispatch idempotent: the same image re-run on another node continues rather than restarts |
| report progress on stdout | logs `Epoch x/y - Loss: ... - Acc: ...% - <s>s` once per epoch | the worker parses the line for live progress, and the trailing seconds is the per-epoch compute the profiler records |
| take config from the environment | reads `EPOCHS_TOTAL`, `BATCH_SIZE` (and `CHECKPOINT_DIR`) | lets the worker set the epoch budget per run — `profilingEpochsNo` for a profile, `epochsTotal` for a standard run |
| use all assigned GPUs | wraps the model in `nn.DataParallel` across the $N$ GPUs the worker exposes | a 2-GPU placement only pays off if the job actually shards its batch over both |
| tolerate `SIGKILL` | no signal handler — a kill just ends the process | stopping a job is plain `docker kill`; safety comes from the epoch-boundary checkpoint, not graceful shutdown |

**Table 3.4:** The training-job contract. An image that ignores it still *runs*, but is not stoppable or resumable: a kill loses all progress and the scheduler cannot migrate it.

**Scope of the built-in loop.** The supplied loop assumes *supervised single-label classification*: it pairs an `Adam` optimiser (lr $= 10^{-3}$) with `CrossEntropyLoss`, feeds each batch as `(inputs, labels)`, and scores top-1 accuracy. A workload that fits this shape needs only the hooks above. One that does not — a regression target, a custom loss, a language-model objective — overrides `train` or `_train_one_epoch` while still inheriting the checkpoint and progress contract, so it stays stoppable and resumable. An image that bypasses `BaseTrainer` altogether still runs, but forfeits those guarantees.

**A complete job.** The following `train.py` is a full, runnable IJM job in its entirety — a CNN on CIFAR-10, using the bundled mixin so only `_create_model` is written:

```
# train.py
import torch.nn as nn
from base import CIFAR10Trainer          # bundled dataset mixin

class Net(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.body = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.ReLU(),
            nn.AdaptiveAvgPool2d(1))
        self.head = nn.Linear(32, num_classes)
```

```
    def forward(self, x):
        return self.head(self.body(x).flatten(1))

class Trainer(CIFAR10Trainer):          # data + batch wiring inherited
    def _create_model(self) -> nn.Module:
        return Net()

if __name__ == "__main__":
    Trainer().train()    # stop/resume/progress: automatic
```

**Packaging and runtime inputs.** The image bundles `base.py` beside the script and runs it as the entrypoint:

```
FROM python:3.13-slim
RUN pip install torch torchvision
COPY base.py train.py ./
CMD ["python", "-u", "train.py"]
```

At dispatch the worker sets `EPOCHS_TOTAL` and `BATCH_SIZE` (a profile run passes `profilingEpochsNo` as `EPOCHS_TOTAL`, Table 3.2) and bind-mounts the per-job checkpoint directory at the default `CHECKPOINT_DIR=/checkpoints`. The legacy P600 node, pinned to CUDA 10.1 / PyTorch 1.5.1, is served by a parallel `Dockerfile.legacy` selected through `IMAGE_TAG_OVERRIDE` (§2.7); the only constraint this adds is that the trainer module must *import* under Python 3.7.

**Datasets.** The bundled mixins fetch MNIST and CIFAR-10 into the shared cache at `/runs/data` (Table 2.5), staged once and mounted read-only on every node. A custom dataset must currently be baked into the image or added to that shared cache: the `directoryToMount` submission field is retained for ANDREAS compatibility but is not bind-mounted by the present worker.

## 3.3 Full API overview

Table 3.5 lists every endpoint. Interactive documentation is available at `/docs` (Swagger UI) and `/redoc` when the API is running.

| Method | Path | Description |
|---|---|---|
| GET | / | Health check endpoint |
| GET | /health | Detailed health check — pings DB and checks job runner |
| GET | /nodes | List all cluster nodes with their current status |
| GET | /gpu-costs | Hourly GPU energy costs used by the scheduler for optimisation |
| GET | /configurations | List all valid hardware configurations in the cluster |
| POST | /jobs | Create a new job |
| GET | /jobs | List all jobs ordered by creation time (paginated via limit/offset) |
| GET | /jobs/{id} | Get a specific job by ID |
| POST | /jobs/{id}/stop | Request job to be stopped (user-driven; sticky PREEMPTED state) |
| POST | /jobs/{id}/resume | Request job to be resumed |
| DELETE | /jobs/{id} | Delete a job, killing any running container first |
| DELETE | /jobs | Delete all jobs and their profiling results |
| GET | /jobs/{id}/logs | Return the output log for a job by proxying to the worker that owns it |
| GET | /profiling-results/{id} | All profiling results for a job, ordered by duration (fastest first) |
| GET | /admin/slots | NodeSlots snapshot: in-memory used vs. DB-authoritative count |
| POST | /admin/reconcile-slots | Force NodeSlots to match DB-authoritative state (idempotent) |
| GET | /admin/dispatch-tasks | List in-flight _dispatch_when_slot_free tasks |

**Table 3.5:** API endpoints.

Three architectural choices set IJM's control plane apart from ANDREAS. First, the *job-management* logic — scheduling, profiling, and dispatch — is consolidated into a single FastAPI service, where ANDREAS used a separate Java Jobs Manager. The cost-aware optimiser itself is *not* reimplemented: IJM consumes the GPUspb solver [FLC+23, FAAG24] — the same line of work as ANDREAS's optimiser — as an external service, so what is consolidated is the job manager, not the optimiser. Second, execution lives in per-node *workers* — also FastAPI — that the API reaches over HTTP through SSH tunnels; that same GPUspb optimiser is reached the same way and owns every standard-run placement decision. Third, IJM exposes explicit `stop`, `resume`, `logs`, and `delete` endpoints, together with `/profiling-results` and `/configurations`, so the user has direct visibility into the scheduler's state.

## 3.4 Runtime environment

Table 3.6 lists the runtime knobs that influence the scheduler. Unless noted otherwise, every variable is read by the API process at startup.

| Variable | Default | Description |
|---|---|---|
| DATABASE_URL | postgresql://.../ijm | PostgreSQL DSN, reached via SSH tunnel |
| OPTIMIZER_URL | unset | The GPUspb HTTP service used for standard-run placement |
| OPTIMIZER_METHOD | RG | GPUspb solver method passed in the request body |
| OPTIMIZER_VERBOSE | 0 | Per-call log verbosity (0–3) |
| PROFILING_CONFIGS_PER_JOB | 2 | How many distinct GPU configurations a single instance profiles before falling through to a standard run |
| IJM_DRIFT_HEARTBEAT_S | 15 | Period of the slot-tracker drift heartbeat that reconciles the in-memory slot count against the database-authoritative count per node |
| WORKER_GPU_MODE | runtime | Worker-side; how Docker exposes GPUs to training containers (runtime / cdi / none) |
| IMAGE_TAG_OVERRIDE | unset | Worker-side; rewrites :latest to the given tag. Used on matemagician to pick the CUDA-10.1-compatible :legacy image |
| HOST_ROOT, HOST_PROJECT_ROOT | /host | Resolve container-internal vs. host paths for Docker bind mounts; HOST_PROJECT_ROOT falls back to HOST_ROOT when unset |

**Table 3.6:** Key environment variables.

# 4. Testing and Evaluation

IJM is verified at two complementary levels, both covered in this chapter: fast *unit and invariant tests* that drive the control plane's atomic claims and recovery paths against *injected* faults, and two *end-to-end scenarios* that run the full job lifecycle on the real two-node cluster. Together they show that IJM realises the optimiser's plan correctly and keeps its slot accounting coherent under both normal and adverse conditions.

## 4.1 Test strategy

The two levels answer different questions. The *unit and invariant tests* (§4.3) ask whether each concurrency invariant of Chapter 2 survives the specific race or failure it is meant to guard against — two claims racing one row, a dropped slot-freed `NOTIFY`, an orphaned dispatch — by injecting that fault directly against a live database and asserting the outcome. The *end-to-end scenarios* (§4.4) ask whether the assembled system realises a real optimiser plan across the whole lifecycle: profile, place on one or two GPUs, preempt, and migrate across nodes — resuming checkpoints across torch versions — all driven only by `POST /jobs` on the two-node cluster (`matemagician`: 2×QuadroP600; `polimi-gpu`: 2×A40). Throughout, the claim is *coherence with the optimiser*, not optimality of the plan itself: placement decisions come from the external GPUspb optimiser (`method=RG`), which IJM consumes but does not implement, and Appendix A documents a *horizon-myopia* in that proxy — orthogonal to IJM's own correctness.

## 4.2 Cluster and job-type setup

The cluster is two profiling-capable nodes, four GPU slots total: `matemagician` with 2×`QuadroP600` and `polimi-gpu` with 2×`A40`. For each new job type the profile sweep exercises all four `(node, GPU-count)` cells; with `PROFILING_CONFIGS_PER_JOB`=2, two instances of the type suffice to cover them. Scenario 1 uses one job type (`cnn_big`, a compute-heavy synthetic CNN); Scenario 2 adds `lstm-small` (a small LSTM on MNIST).

| Type | Architecture | Dataset |
|---|---|---|
| `lstm-small` | LSTM 1-layer, 128 hidden | MNIST (sequence) |
| `lstm-big` | LSTM 3-layer, 256 hidden, dropout 0.2 | MNIST (sequence) |
| `convnet` | 3-layer CNN + BN | CIFAR-10 |
| `efficientnet` | MBConv EfficientNet | CIFAR-10 |
| `cnn_big` | 10-block CNN, 3→ . . . →512 channels | synthetic 128×128×3 |

**Table 4.1:** Training containers under `runtime/`. All share the same checkpoint contract (legacy pre-1.6 `torch.save`, loaded with `strict=False`), so any job can resume across nodes regardless of the destination's torch version.

| Type | A40×1 [s/ep] | A40×2 [s/ep] | QuadroP600×1 [s/ep] | QuadroP600×2 [s/ep] |
|---|---|---|---|---|
| `lstm-small` | 5.88 | 8.61 | 9.17 | 17.71 |
| `cnn_big` | 5.08 | **4.55** | 78.36 | **46.66** |

**Table 4.2:** Steady-state per-epoch compute from `profiling_results` (mean of post-warmup epochs, 2026-06-25 runs; the runtime's own per-epoch timing, after the profiler-measurement fix). Bold marks 2-GPU bundles that beat their 1-GPU counterpart on wall-clock: `cnn_big` wins 1.12× on A40×2 and 1.68× on P600×2; `lstm-small` has no 2-GPU win (DataParallel sync dominates). These are the per-epoch costs IJM measures and the optimiser consumes.

## 4.3 Unit and invariant tests

### 4.3.1 The backend test suite

The control plane carries a pytest suite of over 150 tests under `backend/tests/` covering the data model and request validation, the cluster and cost-table loaders, the scheduling and profiling logic (including the minimum-cost profile-preempt cover), the `NodeSlots` semaphore, and the job-lifecycle endpoints. Most run against in-memory fakes and need no services; the few that must observe real row-locking (`test_sql_integration`, `test_fault_injection`) connect to a live PostgreSQL and *skip* cleanly when none is reachable, so the suite is green offline (`cd backend && uv run pytest`) and exhaustive in CI against the compose stack. The rest of this section covers the fault-injection layer, which turns the concurrency invariants of Chapter 2 from asserted prose into observed recovery.

### 4.3.2 Invariants under injected faults

These tests validate the other half of correctness — that the control plane's concurrency invariants (Chapter 2) hold under *induced* faults, not just on the happy path — at two levels. At the *assertion* level, a fault-injection suite drives the actual conditional-`UPDATE` claims and recovery functions against a live PostgreSQL: it races two placement claims for one row, races two worker Phase-1 claims, clears an assignment underneath an in-flight claim, fires the stuck-dispatch reaper at an orphaned row, and forces the in-memory slot count to disagree with the database in both directions. At the *run* level, a live API instance was driven through the same divergence while `GET /admin/slots` was polled: Figure 4.1 shows the drift heartbeat reconciling in real time — acquiring when the database knows of more occupied slots than memory does, releasing leaked permits when it knows of fewer — with no oversubscription at any point. Table 4.3 maps each invariant to the fault that exercises it and the observed outcome, turning the "coherent slot accounting" row of Table 4.4 from an incidental scenario observation into a direct, repeatable test.

| Invariant | Fault injected | Observed outcome | Checked by |
|---|---|---|---|
| No double placement (atomic claim) | two optimiser applies race one QUEUED row | exactly one claim wins; the other matches zero rows | `test_optimizer_claim_is_exclusive` |
| No duplicate container (worker Phase 1) | two dispatch claims race the same row | exactly one container starts | `test_worker_phase1_claim_is_exclusive` |
| No phantom RUNNING (reaper/claim race) | reaper clears `assigned_node` during a claim | the claim misses; row stays QUEUED, no slot held | `test_reaper_clear_makes_worker_claim_miss` |
| Leaked slot reclaimed (stuck-dispatch reaper) | orphaned QUEUED+assigned row, no live task | assignment reset; permit reclaimed by drift-recover | `test_reaper_resets_orphaned_assignment` |
| Database is authoritative (drift reconcile, both directions) | in-memory `used` forced $\neq$ DB count | heartbeat acquires/releases to the DB count; idempotent | `test_drift_recovers_{leaked,missed}` |
| Capacity never inflates (over-release backstop) | one kill emits two slot-freed NOTIFYs | second release capped and flagged; available $\leq$ total | `test_double_release_is_bounded_and_flagged` |

**Table 4.3:** Each control-plane invariant, the fault injected to exercise it, and the observed outcome. All tests run real SQL against PostgreSQL (`backend/tests/test_fault_injection.py`; the last row lives in `test_node_slots.py`) and skip cleanly when no database is reachable, so they run in CI against the compose stack and stay green offline.

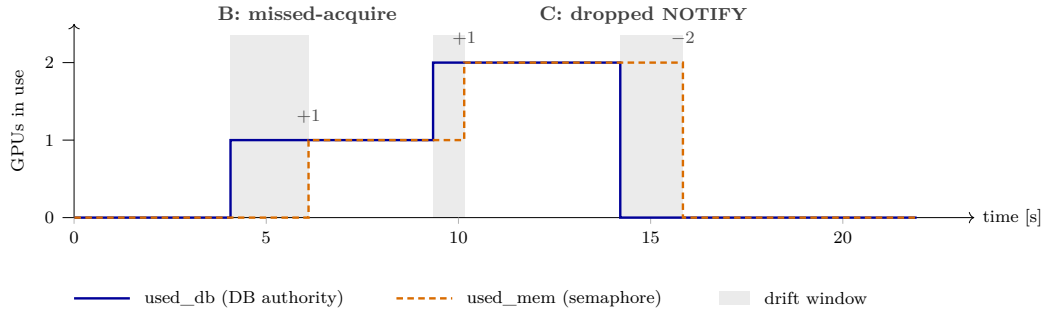


**Figure 4.1:** The drift heartbeat reconciling an injected divergence, from a live API run (`IJM_DRIFT_HEARTBEAT_S`=2, one 2-GPU node). The database count (**solid**) is the authority; the in-memory permit count (**dashed**) follows it. Shaded bands are the windows where the two disagree (`drift`=true). **B**: two RUNNING rows the API never acquired are inserted (an API restart with an in-flight container, or the acquire-side of a lost event); the heartbeat *acquires* (+1, +1). **C**: both rows complete without a `ijm_slot_freed` NOTIFY (the tunnel-reset case); the heartbeat *releases* the two leaked permits (−2). No `SLOT-OVERSUB`/`SLOT-OVERRELEASE` line appears in the log at any point. Artifacts under `documentation/report/e2e_runs/fault_injection/`.

These cover the slot-conservation, atomic-claim, and drift-reconcile invariants. The worker-side *exactly-once* release — the Phase-4 / `_persist_stop` mutual exclusion of §2.3 — and the full cross-node migration path remain exercised by the end-to-end scenarios (§4.4) rather than these unit faults; the table's last row verifies only the API-side *backstop* that caps an over-release, not that mutual exclusion itself.

## 4.4 End-to-end scenarios

Both scenarios below are one end-to-end run each, driven only by `POST /jobs` on the cluster of §4.2; §4.4.1 lists the capabilities they exercise before the per-scenario walkthroughs.

### 4.4.1 Features under test

IJM's job is to realise the optimiser's plan — profile, place, preempt, migrate — correctly and atomically. Table 4.4 lists the capabilities the two scenarios exercise and the observable signal for each; the per-scenario subsections then walk the timelines that produce those signals.

| Feature (IJM realises) | Shown in | How you see it work |
|---|---|---|
| Profile-always sweep | S1, S2 | all four `(node,GPU)` cells measured in `profiles_*.json`; hatched profile bars precede each type's first standard run |
| Single- and dual-GPU placement | S1, S2 | jobs run on ×1 and ×2 bundles (taller "×2" bars); the per-epoch table (Table 4.2) is the 2-GPU speedup IJM exploits |
| Deadline-driven preemption | S2 | the plan drops the cheapest slack job; IJM stops it (`drop-preempt`) and re-queues it — a gap then resume in the timeline |
| Cross-node, cross-version resume | S1, S2 | a job migrates P600→A40 (torch 1.5→2.6); the worker logs "Resumed from epoch $k$" and training continues |
| Coherent slot accounting | S1, S2 | `GET /admin/slots` reports zero oversubscription and zero underflow over the run; the drift heartbeat auto-reconciles |
| Accurate profiling ⇒ prediction | S1, S2 | the optimiser's predicted finish (orange ▼) lands within a few minutes of each bar's real end |

**Table 4.4:** The IJM capabilities the two scenarios exercise, and the observable signal for each. Each scenario is one end-to-end run driven only by `POST /jobs`.

### 4.4.2 Scenario 1 — single type, under-subscribed

We submit five `cnn_big` instances (Table 4.5): four loose-deadline "patient" jobs and one tight-deadline "urgent" job. The cluster is under-subscribed when URGENT arrives (both P600 slots free), so this run exercises the profile sweep, single- and dual-GPU placement, and a cross-node migration with cross-version resume — but no preemption (Scenario 2 adds that).

| Label | Type | Priority | Deadline | Epochs | Submitted at |
|---|---|---|---|---|---|
| JOB1 | `cnn_big` | 4 | +30 min | 75 | $t = 0$ s |
| JOB2 | `cnn_big` | 4 | +30 min | 75 | $t = 5$ s |
| JOB3 | `cnn_big` | 2 | +2 h | 50 | $t = 10$ s |
| JOB4 | `cnn_big` | 1 | +2 h | 25 | $t = 15$ s |
| URGENT$^\star$ | `cnn_big` | 5 | +10 min | 200 | $t$=7.5 min |

**Table 4.5:** Scenario 1 submissions: four patients and one urgent job.

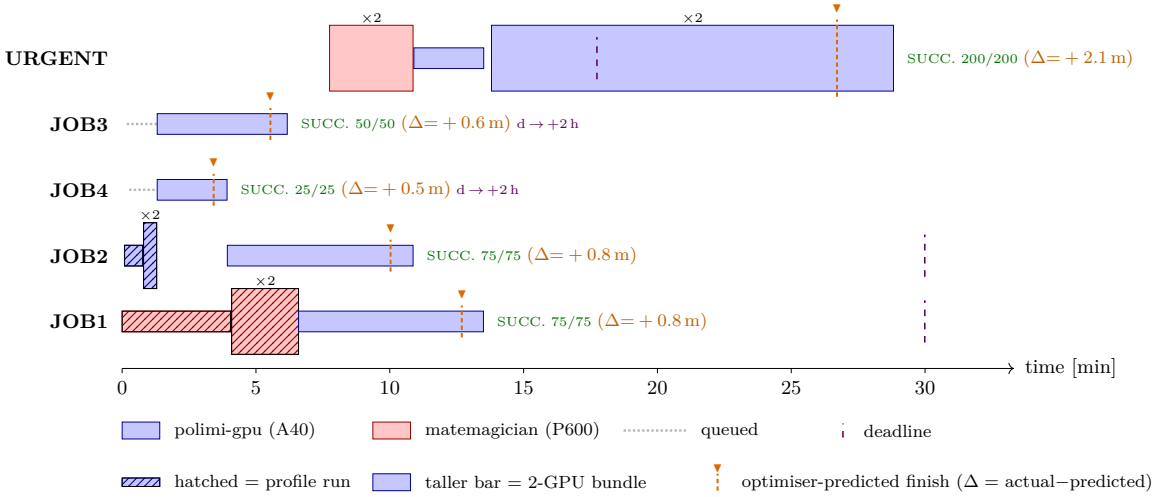


**Figure 4.2:** Scenario 1 timeline (run 2026-06-25, $t$=0 at first dispatch). Hatched bars are the profile sweep (all four cells per type); taller "×2" bars are dual-GPU runs. URGENT lands on the free P600×2 (the whole `matemagician` node is idle; no patient is preempted), then migrates onto A40 — first A40×1, then A40×2 — as the two prio-4 patients finish, resuming its checkpoint across the torch 1.5→2.6 boundary each time. The orange ▼ is the optimiser's predicted finish (Δ = actual−predicted on the status label): after the profiler-measurement fix every Δ is within a few minutes (the four patients +0.5 to +0.8 min; URGENT +2.1 min, the one-off cross-node resume cost). Why URGENT sits on the slow P600 rather than faster A40 is the proxy's horizon-myopia (Appendix A).

| $t$ [min] | Job(s) | What IJM does |
|---|---|---|
| 0.0 | J1, J2 | J1 begins the P600 profile sweep; J2 the A40 sweep. "Profile-always" gate: neither is yet a standard candidate. |
| 1.3 | J3, J4 | Once the type's A40 cells are measured, prio-2 J3 (50 ep) and prio-1 J4 (25 ep) are placed standard on A40×1 each. |
| 3.9 | J4, J2 | J4 ends naturally (25/25); J2's sweep finishes, so J2 (prio 4) takes the freed A40×1 — no eviction. |
| 6.2 | J3 | J3 ends naturally (50/50). |
| 6.6 | J1 | J1's sweep finishes; J1 (prio 4) takes the freed A40×1. Cluster: J1, J2 on A40×1; **both P600 slots free**. |
| **7.7** | **URGENT** | **Submitted (prio 5, +10 min, 200 ep). The optimiser places it on the free P600×2; IJM dispatches it with no preemption** (the node is idle, `drop-preempt` empty). Why the slow P600 and not faster A40: Appendix A. |
| **10.9** | **URGENT** | **J2 ends (75/75); IJM migrates URGENT P600×2 → A40×1** onto the freed slot — a cross-node, cross-version move (`drop-preempt` empty), checkpoint resumed on the A40. |
| **13.5** | **URGENT** | **J1 ends (75/75); IJM migrates URGENT A40×1 → A40×2**, both A40 GPUs now free. Again no patient is displaced. |
| 17.7 | URGENT | Deadline (violet tick on Fig. 4.2); URGENT is ∼40% through, on A40×2. |
| 28.8 | URGENT | URGENT ends (200/200), ∼11 min past deadline — 200 epochs cannot fit the +10-min deadline on *any* bundle. |

**Table 4.6:** Scenario 1 decision log (2026-06-25). Every dispatch and migration is IJM realising the optimiser's plan.

This run exercises four of the six features: the profile sweep (all four `cnn_big` cells measured), single- and dual-GPU placement (A40×1, P600×2, A40×2), two cross-node cross-version migrations (P600→A40×1→A40×2, each resuming the checkpoint across torch versions), and accurate prediction (Fig. 4.2). The slot tracker stayed coherent throughout: `/admin/slots` reported zero oversubscription and zero underflow, with the drift heartbeat

auto-reconciling. The one decision that looks odd — URGENT on slow P600 — is the upstream proxy's choice, not a placement IJM made; it realised that plan faithfully and then migrated URGENT onto A40 the moment a slot freed.

### 4.4.3 Scenario 2 — two types, full cluster

Scenario 2 fills the cluster and adds a second type (Table 4.7): two prio-5 `cnn_big` pins occupy A40, two slack prio-1 `lstm-small` patients occupy P600. An urgent `cnn_big` then arrives, so this run additionally exercises *deadline-driven preemption* and a second cross-node migration — including resuming the *evicted* job.

| Label | Type | Priority | Deadline | Epochs |
|---|---|---|---|---|
| PIN-A | B (`cnn_big`) | 5 | +10 min | 80 |
| PIN-B | B (`cnn_big`) | 5 | +10 min | 80 |
| P-LSTM-1 | S (`lstm-small`) | 1 | +8 h | 200 |
| P-LSTM-2 | S (`lstm-small`) | 1 | +8 h | 200 |
| URGENT⋆ | B (`cnn_big`) | 5 | +22 min | 40 |

**Table 4.7:** Scenario 2 submissions: two A40 pins, two slack P600 patients, and one urgent job.

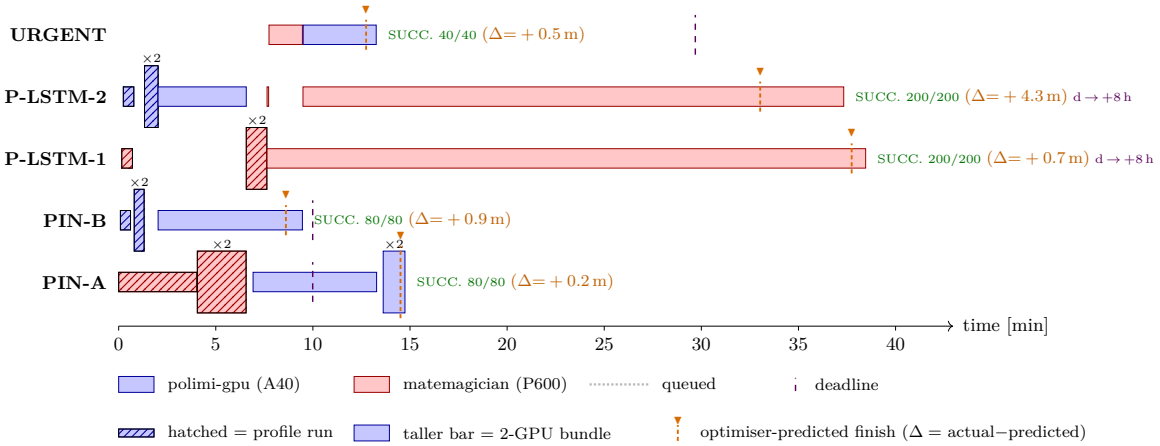


**Figure 4.3:** Scenario 2 timeline (run 2026-06-25, $t$=0 at first dispatch). The cluster is full when URGENT arrives, so the plan *preempts* a single prio-1 `lstm-small` (P-LSTM-2): IJM stops it and places URGENT on P600×1, then migrates URGENT onto A40×1 the moment PIN-B ends, where it meets its deadline; the evicted patient resumes on P600 — the gap-then-resume visible on its row. Predicted-finish overlay as in Fig. 4.2: most jobs land within ∼1 min, the outlier being P-LSTM-2 at +4.3 min — preempted and then resumed sharing `matemagician`, its final run hit co-location contention (∼10.6 s/epoch vs. the solo profile's ∼9.2 s) a solo profile cannot foresee. Which preemptible bundle the plan picks is the proxy's choice (Appendix A).

| $t$ [min] | Job(s) | What IJM does |
|---|---|---|
| 0.0–6.6 | all four | Profile sweep over both types' eight (type, GPU-bundle) cells. PIN-A carries the P600 sweep; PIN-B + P-LSTM-2 carry the A40 sweep; P-LSTM-1 atomic-claims the remaining cells. |
| 2.0 | PIN-B, P-LSTM-2 | Both finish their profile budgets; placed standard on A40×1 each (only profiled bundles available). |
| 6.9 | PIN-A, P-LSTM-2 | PIN-A's sweep finishes; the plan prefers prio-5 PIN-A over prio-1 P-LSTM-2 on A40 ⇒ IJM *drop-preempts* P-LSTM-2; PIN-A on A40×1. |
| 7.6 | P-LSTM-1, P-LSTM-2 | P-LSTM-1 and the just-evicted P-LSTM-2 placed on matemagician P600×1 each. Cluster now **full**: A40 = two prio-5 pins, P600 = two prio-1 lstm. |
| **7.7** | **URGENT** | **Submitted (prio 5, +22 min, 40 ep). The plan drop-preempts *one* prio-1 lstm (P-LSTM-2) and places URGENT on P600×1; IJM stops the lstm and dispatches URGENT.** A40 is held by equal-priority pins; which P600 bundle is chosen: Appendix A. |
| **9.5** | **URGENT, P-LSTM-2** | **PIN-B ends (80/80); IJM migrates URGENT P600×1 → A40×1** (faster, now free); the evicted P-LSTM-2 *resumes* on P600×1. |
| 13.3 | URGENT | URGENT ends (40/40) on A40×1, ∼16 min *inside* its +22-min deadline. |
| 14.7 | PIN-A | PIN-A ends (80/80); the second A40 frees. |
| ∼38 | P-LSTMs | Both lstm-small patients finish on P600×1 (their +8-h deadlines are far off). |

**Table 4.8:** Scenario 2 decision log (2026-06-25).

Scenario 2 adds the two features S1 could not show. *Preemption*: the plan omits the cheapest slack job, and IJM stops P-LSTM-2, re-queues it, and resumes it once a slot reopens — the gap-then-resume on its row. *Resume of a migrated/evicted job*: both URGENT (P600→A40) and the evicted patient come back from their checkpoints with no lost epochs. As in Scenario 1 the slot tracker stayed coherent (zero oversubscription, zero underflow), and URGENT met its deadline — here by migrating onto A40 once a pin finished. The placement choices themselves (P600×1, evict one) follow the proxy's cost minimisation (Appendix A); IJM's contribution is realising them without dropping a slot or losing progress.

## 4.5 Reproducibility

All figures and tables derive from the 2026-06-25 run snapshots committed under `documentation/report/e2e_runs/2026-06-25/{s1,s2}/`:

- **Committed artifacts**: `api.log`, `jobs.json`, `profiles_*.json`, `chart_s{1,2}.tex`.
- **Per-epoch costs**: `profiling_results.duration_seconds` (Table 4.2). These runs use the corrected profiler — the worker records the runtime's own per-epoch compute time (warmup excluded), not log-line arrival intervals (`worker/profiling.py`, `execution.py`).
- **Charts**: auto-generated from the API log by `generate_chart_tex.py` (bars, deadline ticks, and the predicted-finish overlay); every bar is a real log event.
- **Drivers**: `infra/e2e_scenario.sh`, `infra/e2e_scenario_2types.sh`; slots at `GET /admin/slots`; API logs via `docker logs ijm-api` (stack up by `infra/ijm up`).
- **Fault-injection evidence** (§4.3.2): the invariant tests in `backend/tests/test_fault_injection.py` and the live drift-reconcile run under `documentation/report/e2e_runs/fault_injection/` (`run.py`, `slots_timeline.csv`, `api.log`; Figure 4.1 generated by `infra/generate_fault_chart.py`).

# 5. Conclusion

## 5.1 Summary of contributions

IJM is an independent reimplementation of the ANDREAS approach as a distributed control plane: one FastAPI service drives per-node workers over SSH-tunnelled HTTP, and an external GPUspb optimiser is consumed as a peer service rather than embedded in the scheduler. On that topology the report makes three contributions. First, a *profile-always* policy keyed by the unique index on `(job_id, gpu_config)`, so profile measurements are atomically shared across instances of the same job type. Second, *cross-version checkpoint resume* between PyTorch 1.5.1 (CUDA 10.1, `matemagician`) and PyTorch 2.6 (CUDA 12.4, `polimi-gpu`), letting an in-flight job migrate between heterogeneous GPU generations without losing progress (§2.7). Third, a control plane that realises the optimiser's plan *coherently* — profiling, single- and dual-GPU placement, deadline-driven preemption, and cross-node migration, with slot accounting that stays free of oversubscription and underflow — demonstrated end-to-end by the two scenarios (Chapter 4). The claim throughout is *coherence*, not optimality: the scenarios show that IJM faithfully realises whatever plan the optimiser returns. Benchmarking that plan against FIFO, EDF, or priority-only scheduling — or assessing the optimiser's own solution quality — is deliberately out of scope, a concern of the upstream optimiser [FLC+23, FAAG24] rather than of the control plane built here.

Along the way the scenarios also surface a *horizon-myopia* in the upstream GPUspb `method=RG` proxy: tardiness scored at the next-event horizon lets an urgent job be parked on slow-but-free hardware, and on a full cluster the same myopia picks the cheapest preemptible bundle rather than the least-tardy one. This is a property of the optimiser we consume, not of IJM (Appendix A); IJM stayed correct regardless — it realised whatever plan the proxy returned, and the urgent job still met its deadline via migration.

## 5.2 Limitations

The reference deployment uses GPUspb's `method=RG`, whose proxy is horizon-myopic; the deadline-aware `method=PR` variant (Path Relinking [FLC+23]) exists upstream but is not the default and has not been validated end to end against the IJM control plane. The control-plane invariants are now backed by a real-SQL fault-injection suite (§4.3.2), but worker-side coverage is still almost entirely end-to-end — `worker/tests/` holds only a regression test for the per-epoch profiler measurement, so the worker's exactly-once release and the cross-node migration path are validated only by the scenarios; only two GPU generations (A40 and QuadroP600) have been validated; and the cluster used throughout Chapter 4 is two nodes with four GPU slots in total, so the scaling properties of the four-phase pass at tens of nodes are an open question. Finally, cross-node resume assumes a shared filesystem across the nodes (here rclone-over-SFTP, §2.7) — a deployment dependency rather than a property of the control plane itself.

## 5.3 Future work

Two follow-ups have concrete entry points in the codebase. First, re-run Scenario 1 with `OPTIMIZER_METHOD=PR`: `proxy_function_max` is per-job deadline-aware, so the prediction from §A.3 is that URGENT is rushed onto the fastest hardware (preempting the prio-4 patients) from round 1, rather than parking on the slow free P600 first and finishing ∼11 min

late. Second, extend the two-node evaluation to a larger cluster once a third GPU class is available; this is where the profile-claim's atomicity and the optimiser-call latency (currently a 30 s timeout) will be stressed.

# A. The GPUspb proxy and its horizon-myopia

IJM delegates standard-run placement to the external GPUspb optimiser [FLC+23, FAAG24] (run with `method=RG`); it does *not* implement the optimiser. This appendix documents what that proxy minimises and a *horizon-myopia* in its objective that explains several placements in Chapter 4 — in particular why an urgent job is sometimes parked on slow-but-free hardware. The behaviour is a property of the upstream proxy, not of IJM, which faithfully realises whatever plan the optimiser returns.

## A.1 Objective and cost model

Per scheduling decision, Random Greedy scores a candidate plan with the proxy `proxy_function_min` (`optimizer.cpp`):

$$\begin{aligned}\textsc{Cost}_{\mathrm{RG}} = &\sum_{\text{sched } j} \Big( r(b_j)\,\Delta t + w(p_j)\,[\,t_0{+}\Delta t - d_j\,]^+ \Big) + \sum_n \texttt{cost}_n\,\Delta t \\ &+ \lambda \sum_{\text{unsched } j} w(p_j)\,[\,t_0{+}T_j^{\text{full}} - d_j\,]^+,\end{aligned}$$

with indices $j$ (over the plan's scheduled/unscheduled jobs) and $n$ (over the cluster's nodes), and parameters ($[x]^+ \equiv \max(0, x)$):

- $\Delta t$ — next-event horizon: the soonest finish among the plan's scheduled jobs;
- $t_0$ — the current time;
- $b_j$, $p_j$, $d_j$ — job $j$'s assigned GPU bundle, priority, and deadline;
- $r(b)$ — hourly rate of GPU bundle $b$ (Table A.2, left);
- $w(p)$ — per-priority tardiness weight (Table A.2, centre);
- $\texttt{cost}_n$ — per-node operating cost from `nodes_config.json` (Table A.2, right), passed through in the optimiser request. Note the sum is over *every* configured node: `compute_nodeCost` charges $\texttt{cost}_n\Delta t$ whether or not the node hosts a job (no occupancy filter), so this term gives no consolidation incentive;
- $T_j^{\text{full}}$ — job $j$'s full remaining run time on its *slowest profiled bundle* (the max over the job's measured configs, `get_maxExecTime`), charged in full and *not* clipped to $\Delta t$;
- $\lambda$ — unscheduled-job penalty multiplier, hard-coded to $\lambda$=100 in `proxy_function_min`.

Three features of this objective (Table A.1) drive everything the scenarios show.

| Proxy feature | Charged | Consequence |
|---|---|---|
| GPU/tardiness/node of *scheduled* jobs | only to horizon $\Delta t$ | a placement that overruns a far-off deadline still reads zero tardiness *now* (horizon-myopia; Scenario 1) |
| *Unscheduled* job | $\lambda$=100× *full* worst-case tardiness | omitting a tight high-priority job is ruinous, a slack low-priority one nearly free (drop-the-cheapest; Scenario 2) |
| Migration / preemption | no term | the plan is scored alone, ignoring what runs — in-proxy, eviction is free |

**Table A.1:** The three features of $\text{Cost}_{\text{RG}}$ that drive both scenarios.

$T_j^{\text{full}}$ is the full run on a job's *slowest profiled* bundle, so it jumps as the sweep measures slower hardware (`QuadroP600` is ∼15× slower than `A40` for `cnn_big`, Table 4.2): a long tight-deadline job can flip from free-to-omit to *must-schedule* with no change in free capacity, which Scenario 1's sweep (§4.4.2) exercises.

| GPU bundle | $/hour |
|---|---|
| `QuadroP600×1` | 0.030 |
| `QuadroP600×2` | 0.055 |
| `A40×1` | 0.300 |
| `A40×2` | 0.550 |

| Priority | TardWeight ($/sec) |
|---|---|
| 1 | 0.000254 |
| 2 | 0.000301 |
| 3 | 0.000349 |
| 4 | 0.000396 |
| 5 | 0.000444 |

| Node | $\texttt{cost}_n$ ($/h) |
|---|---|
| `matemagician` | 0.10 |
| `polimi-gpu` | 0.30 |

**Table A.2:** Bundle hourly rates (left), per-priority tardiness weights (centre), and per-node operating cost $\texttt{cost}_n$ (right).

| Type | A40×1 [$/ep] | A40×2 [$/ep] | P600×1 [$/ep] | P600×2 [$/ep] |
|---|---|---|---|---|
| `lstm-small` | 0.000490 | 0.001315 | 0.0000764 | 0.000271 |
| `cnn_big` | 0.000423 | 0.000695 | 0.000653 | 0.000713 |

**Table A.3:** GPUcost per epoch, computed as per-epoch[s] × hourly-rate (Table A.2) / 3600. Multiply by `epochsTotal` to obtain the optimiser's energy term for a plan that runs the job to completion on the listed bundle.

## A.2 Judging the proxy: an idealised yardstick

$\text{Cost}_{\text{RG}}$ is what the optimiser minimises, but it is a *per-decision* surrogate. To judge its choices we compare against an *idealised* full-run cost — the same terms, but with tardiness (and GPU/node cost) measured at each job's *real* completion instead of at $\Delta t$. The gap between the two is what the scenarios expose:

- **Horizon-myopia (free slot ⇒ Scenario 1).** Charged only to $\Delta t$, tardiness reads zero while a deadline is still in the future at the horizon — so the proxy parks an urgent job on a cheap, free, far-too-slow bundle rather than faster hardware it would have to clear first.

- **Drop-the-cheapest (full cluster ⇒ Scenario 2).** When jobs exceed capacity, every plan leaves someone unscheduled, and the $\lambda$=100 penalty drops whoever is cheapest to omit — smallest $w(p)\,[\text{lateness}]^+$. On the true per-epoch costs the same myopia also governs *which* preemptible bundle then absorbs the urgent job (the cheapest, not the least-tardy), so Scenario 2 exhibits both effects.
- **Preemption is a side effect.** The objective is stateless: the optimiser picks a target plan, and IJM realises it by stopping whatever the plan dropped — the optimiser never reasons about preemption.
- **`method=PR` would differ.** The deadline-aware variant scores per-job *real* finish times and would dodge the horizon-myopia.

## A.3 URGENT placement under the proxy — Scenario 1

| Bundle | per-epoch [s] | GPUcost/ep [$] | 50-ep [$] | 200-ep [$] |
|---|---|---|---|---|
| A40×1 | 5.08 | 0.000423 | 0.0212 | **0.0847** |
| A40×2 | 4.55 | 0.000695 | 0.0348 | 0.1390 |
| QuadroP600×1 | 78.36 | 0.000653 | 0.0327 | 0.1306 |
| QuadroP600×2 | 46.66 | 0.000713 | 0.0356 | 0.1426 |

**Table A.4:** Per-bundle data the optimiser sees for `cnn_big` in Scenario 1. Full-run (200-ep) energy is now cheapest on A40×1 (bold), not P600 — P600 is ∼15× slower, outweighing its ∼10× cheaper hourly rate — yet the proxy still picks P600 (Table A.5): it bills scheduled jobs only to the horizon at the hourly rate, not to real completion.

At URGENT's submission ($t$≈7.7 min) the whole P600 node is free, and `proxy_function_min` ranks the plan that drops URGENT onto that free P600×2 as *cost-minimal* — the run's `OPT RESP` logs its cost at $0.030 — even though that bundle overruns the +10-min deadline by ∼2.4 h. Table A.5 shows why: the proxy charges every job only up to the horizon $\Delta t$ = `first_finish_time` (the soonest finish in the plan), and in every cheap plan a prio-4 patient finishes first a few minutes out — so URGENT's tardiness, read at that horizon, is exactly zero on every bundle except the one that makes URGENT itself finish first.

| Plan for URGENT | ExecTime [s] | meets +10 min? | in-proxy Tardy [s] | Proxy verdict |
|---|---|---|---|---|
| A40×2 | 910 (15 min) | no | 310 | `get_best_setup`'s own pick (fastest, since no bundle is deadline-reachable), but RG rejects it: URGENT becomes the earliest finish (patients displaced to slow P600), stretching $\Delta t$ and finally exposing its own tardiness ⇒ dearest plan |
| A40×1 | 1 016 (17 min) | no | 0 | tardiness ties the chosen plan, but the A40 hourly rate is 10× P600's, so over the horizon it scores dearer — never preferred *for URGENT* |
| P600×1 (free) | 15 672 (261 min) | no | 0 | near-ties the chosen plan (also free, cheapest rate, tardiness masked); RG took the 2-GPU bundle of the same idle node |
| **P600×2 (free node)** | **9 332 (155 min)** | **no** | **0** | **✓ chosen (cost-minimal): the whole P600 node is idle, a prio-4 patient sets the ∼3 min horizon, so URGENT's 2.4-h overrun stays invisible and its GPU is billed for only ∼3 min of cheap P600** |

**Table A.5:** URGENT plan comparison, Scenario 1 — the values `proxy_function_min` actually scores at submission. ExecTime = 200× per-epoch (Table 4.2); no bundle reaches the +10-min deadline, so `get_best_setup` would take the fastest (A40×2), but the full-plan proxy overrides it. in-proxy Tardy = $[\,t_0+\Delta t-d\,]^+$ with $\Delta t$ = `first_finish_time`: zero whenever a quick patient (not URGENT) sets $\Delta t$, positive only in the A40×2 plan where URGENT itself finishes first. The chosen plan's logged proxy cost is \$0.030.

Under a *deadline-aware* objective — one that scored each job's *real* finish time, as `method=PR`'s `proxy_function_max` does — this ranking inverts: A40×2 becomes best while the slow free-P600 bundles fall to the bottom. Table A.6 gives that counterfactual full-run yardstick; it is *not* what the reference `method=RG` deployment minimises — by it, the run's P600×2 choice costs ∼13× the ideal A40×2.

| Plan for URGENT | ExecTime [s] | Tardy [s] | GPUcost [\$] | NodeCost [\$] | TardCost [\$] | **Total [\$]** | rank |
|---|---|---|---|---|---|---|---|
| A40×2 | 911 | 311 | 0.139 | 0.101 | 0.138 | **0.378** | **1 (best)** |
| A40×1 | 1 016 | 416 | 0.085 | 0.113 | 0.185 | 0.383 | 2 |
| P600×2 (run's choice) | 9 332 | 8 732 | 0.143 | 1.037 | 3.877 | 5.057 | 3 |
| P600×1 | 15 672 | 15 072 | 0.131 | 1.741 | 6.692 | 8.564 | 4 (worst) |

**Table A.6:** *Counterfactual* idealised full-run cost — *not* the deployed objective. Same terms as `proxy_function_min` but with tardiness, GPU, and node cost measured at each job's *real* completion instead of at $\Delta t$. Tardy = max(ExecTime − 600, 0); TardCost = Tardy $\times w_5$ ($w_5$=0.000444 \$/s); NodeCost ≈ ($\texttt{cost}_{\text{mate}}$+$\texttt{cost}_{\text{polimi}}$) × ExecTime/3600. This is how a deadline-aware proxy *would* rank the plans — A40×2 best; the run took P600×2 (3rd of 4, ∼13× the ideal), because that minimised the horizon-myopic proxy actually deployed (Table A.5), not this yardstick.

`method=PR` (with `proxy_function_max`, deadline-aware per job) would have seen URGENT's true finish time from the first round and rushed it onto A40×2 immediately — preempting both patients up front and finishing ∼5 min late instead of ∼11 — but we keep `method=RG` as the reference deployment.

## A.4 URGENT placement under the proxy — Scenario 2

With the cluster full, the corrected per-epoch costs leave *no* preemptible bundle able to meet URGENT's +22-min deadline, so the same myopia simply takes the cheapest preemptible bundle.

| Plan | ExecTime | meets +22 min? | GPUcost [$] | Decision |
|---|---|---|---|---|
| A40×1 (evict 1 prio-5 pin) | 3.4 min | yes | 0.0171 | off-limits: would evict an equal-priority pin, which URGENT cannot preempt |
| A40×2 (evict 2 prio-5 pins) | 3.3 min | yes | 0.0298 | off-limits: same, for *both* equal-priority pins |
| P600×2 (evict 2 prio-1 lstm) | 31 min | no | 0.0285 | rejected: dearer 2-GPU rate than P600×1 over the horizon, and evicts a second patient, for no proxy gain (tardiness masked either way) |
| **P600×1 (evict 1 prio-1 lstm)** | **52 min** | **no** | **0.0261** | ✓ **chosen**: the only deadline-meeting bundles (A40) are off-limits, so RG takes the cheapest *preemptible* one; horizon-myopia reads URGENT's tardiness as zero, so the 52-min overrun is invisible |

**Table A.7:** URGENT plan comparison, Scenario 2. ExecTime = 40× per-epoch (Table 4.2); GPUcost is the full-run energy. Both deadline-meeting bundles are A40, held by equal-priority pins URGENT cannot preempt; among the *preemptible* (P600) bundles neither meets the +22-min deadline, so there is no deadline-reachable option for `get_best_setup` and RG falls back to plain cost minimisation — with tardiness masked by horizon-myopia, that is simply the cheapest P600 bundle, P600×1. URGENT nonetheless *meets* its deadline by migrating onto the freed A40×1 once a pin finishes (Chapter 4, Scenario 2).

# Bibliography

[AF+21] Danilo Ardagna, Federica Filippini, et al. ANDREAS – artificial intelligence training scheduler for accelerated clusters. Technical report, Politecnico di Milano / TETRAMAX, 2021. Horizon 2020 project deliverables D1–D3.

[FAAG24] Federica Filippini, Jonatha Anselmi, Danilo Ardagna, and Bruno Gaujal. A stochastic approach for scheduling AI training jobs in GPU-based systems. *IEEE Transactions on Cloud Computing*, 12(1):53–69, 2024.

[FLC+23] Federica Filippini, Marco Lattuada, Michele Ciavotta, Arezoo Jahani, et al. A path relinking method for the joint online scheduling and capacity allocation of DL training workloads in GPU as a service systems. *IEEE Transactions on Services Computing*, 16(3):1630–1646, 2023.